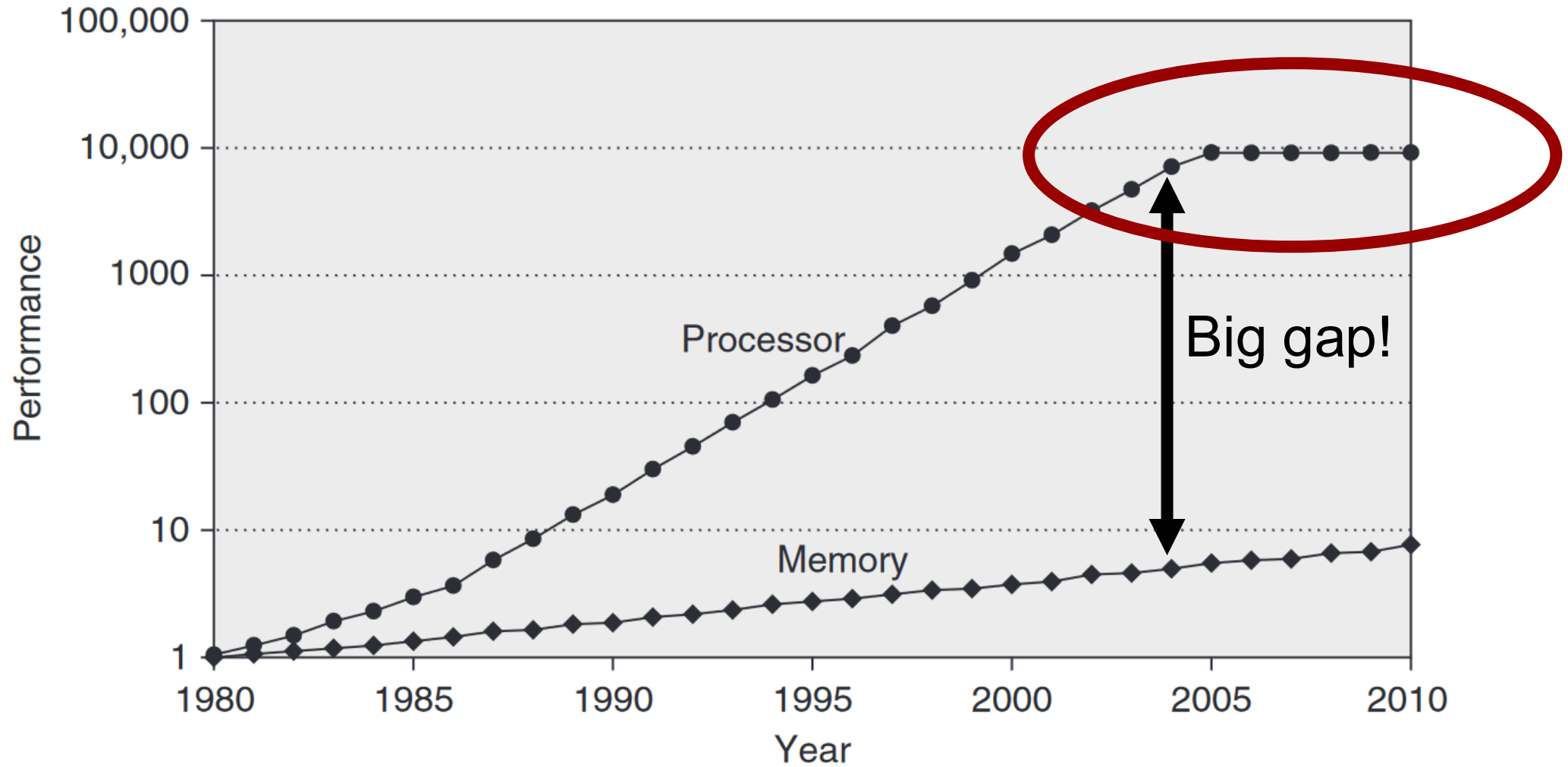


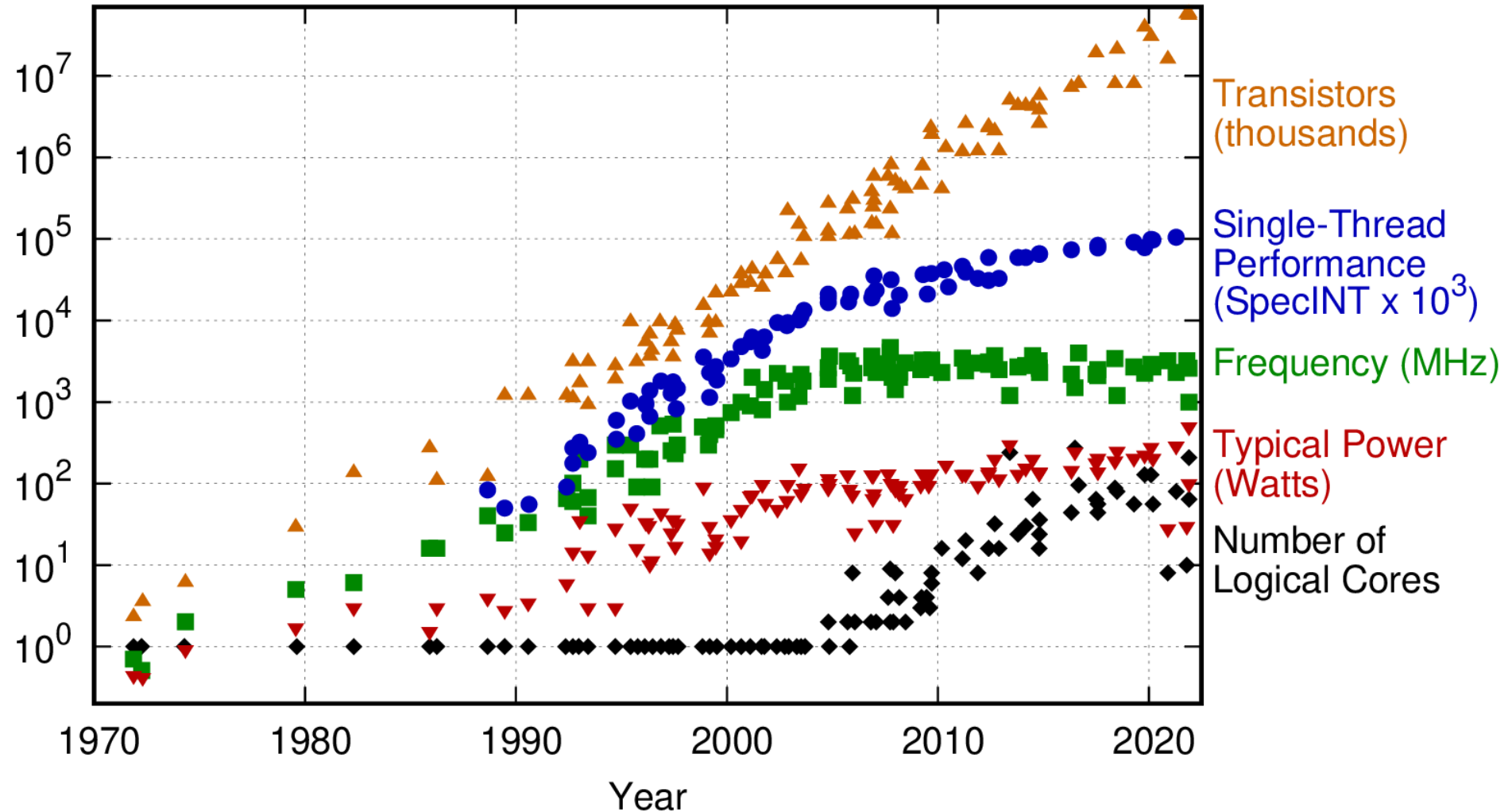
Week 5 – Parallelism Part 1

02613 Python and High-Performance Computing



Hennessey & Patterson, "Computer Architecture: A Quantitative Approach", 5th Ed.

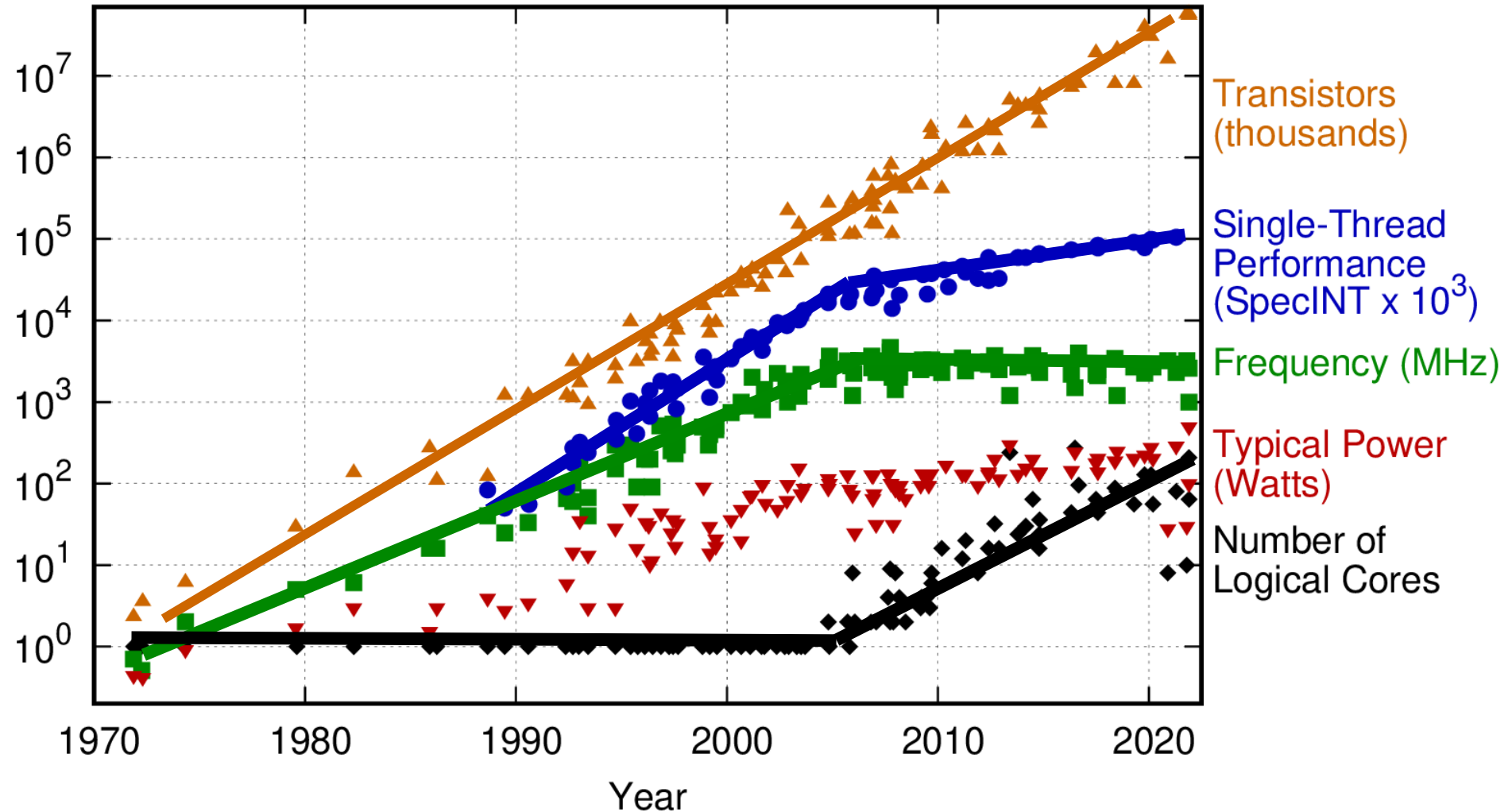
50 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

<https://github.com/karlrupp/microprocessor-trend-data>

50 Years of Microprocessor Trend Data



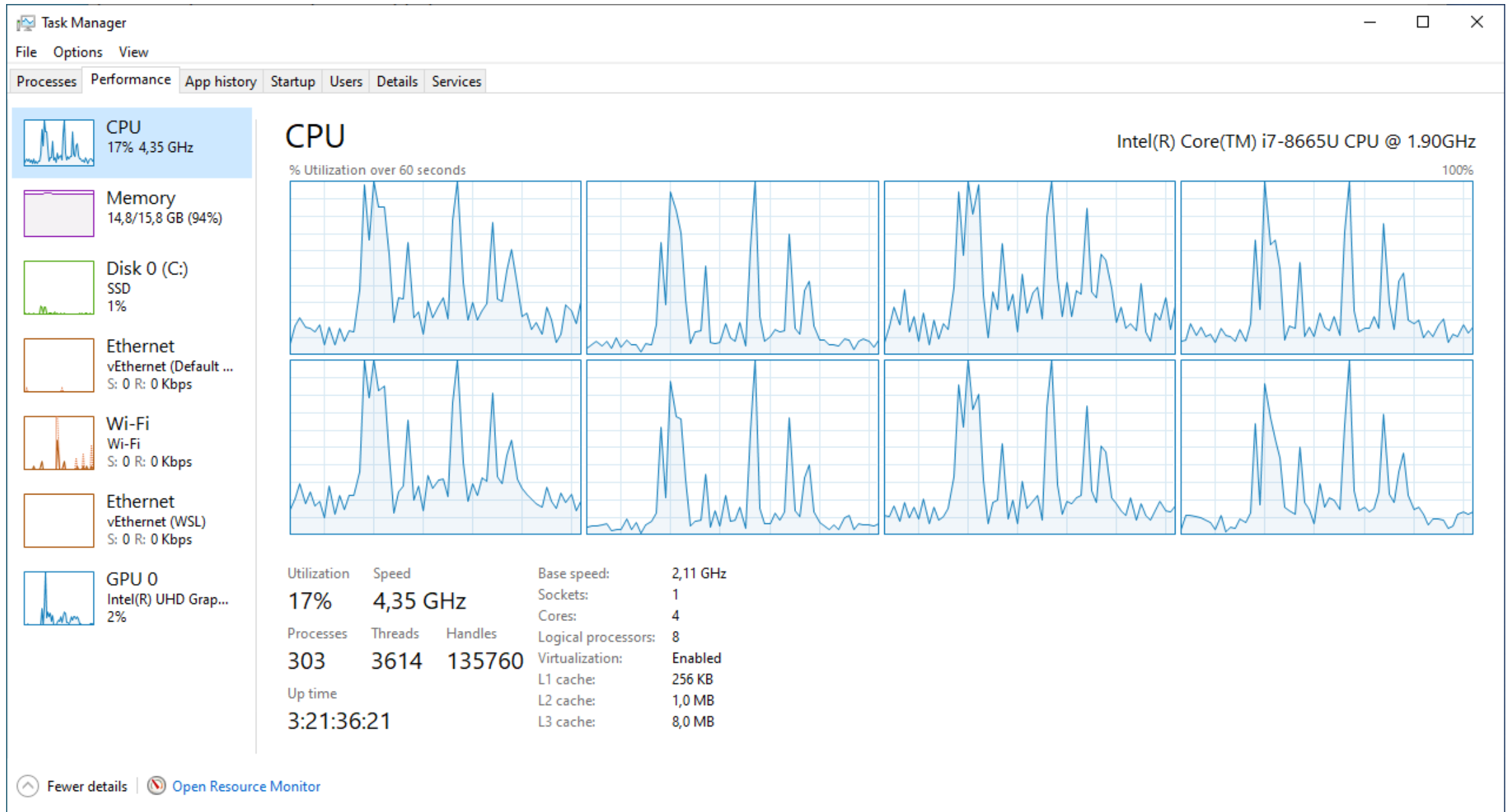
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

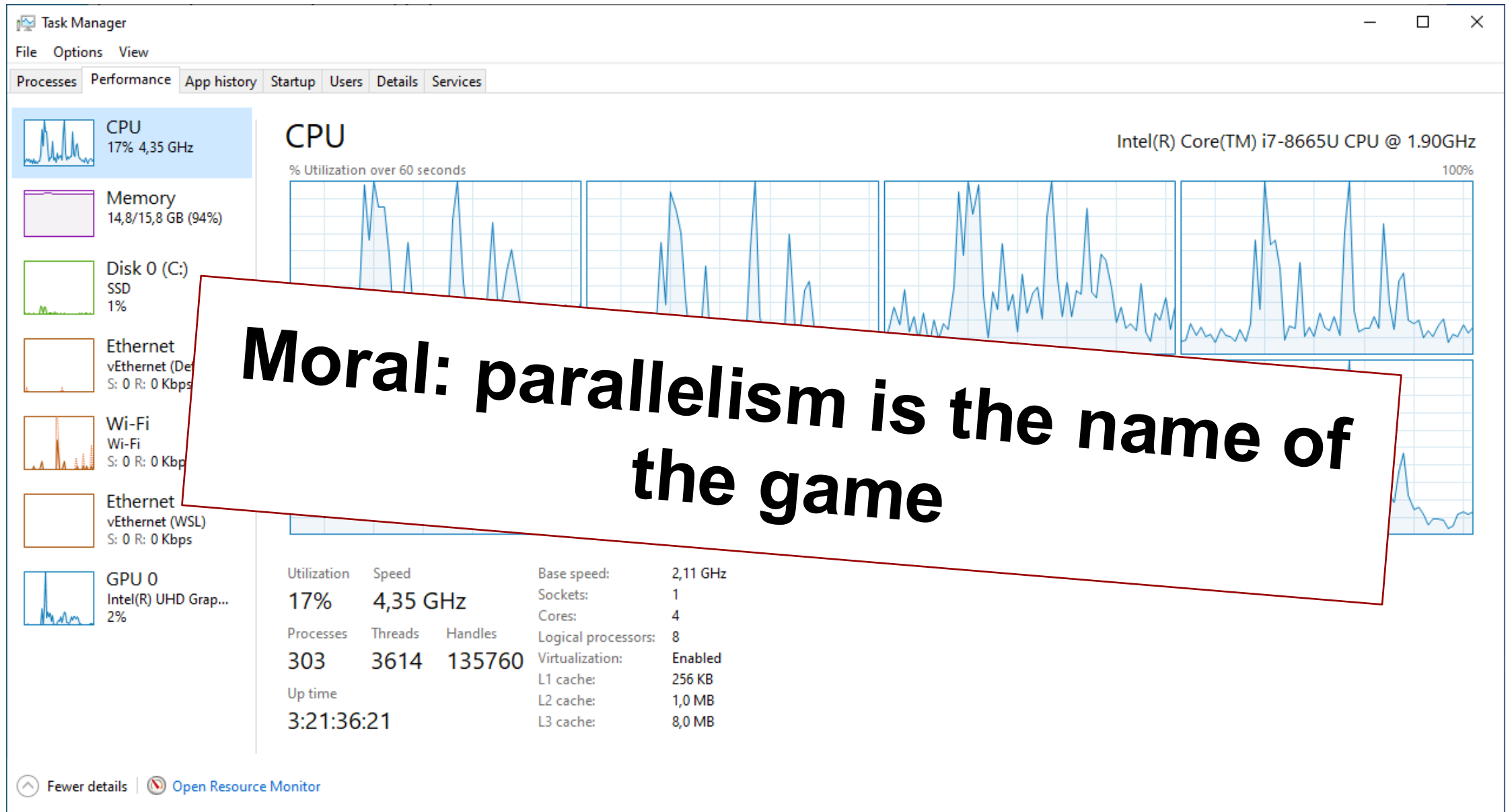
<https://github.com/karlrupp/microprocessor-trend-data>

```

1 [ 0.0%] 13 [ 0.7%] 25 [ 0.0%] 37 [ 0.7%]
2 [ 0.0%] 14 [ 0.0%] 26 [ 0.0%] 38 [ 0.0%]
3 [ 0.0%] 15 [ 0.0%] 27 [ 0.0%] 39 [ 0.0%]
4 [ 0.0%] 16 [ 0.0%] 28 [ 0.0%] 40 [ 0.7%]
5 [ 0.0%] 17 [ 0.0%] 29 [ 0.0%] 41 [ 0.0%]
6 [ 0.0%] 18 [ 0.7%] 30 [ 0.0%] 42 [ 0.0%]
7 [ 0.0%] 19 [ 2.0%] 31 [ 0.0%] 43 [ 0.0%]
8 [ 0.0%] 20 [ 0.0%] 32 [ 0.0%] 44 [ 0.0%]
9 [ 0.0%] 21 [ 0.0%] 33 [ 0.0%] 45 [ 0.7%]
10 [ 0.0%] 22 [ 0.0%] 34 [ 0.0%] 46 [ 0.0%]
11 [ 0.0%] 23 [ 0.0%] 35 [ 0.0%] 47 [ 0.7%]
12 [ 0.0%] 24 [ 0.0%] 36 [ 0.0%] 48 [ 0.0%]
Mem[|||||] 4.03G/377G] Tasks: 95, 195 thr; 1 running
Swp[ 0K/20.0G] Load average: 0.24 0.16 0.09
Uptime: 27 days, 11:44:00

```





Today

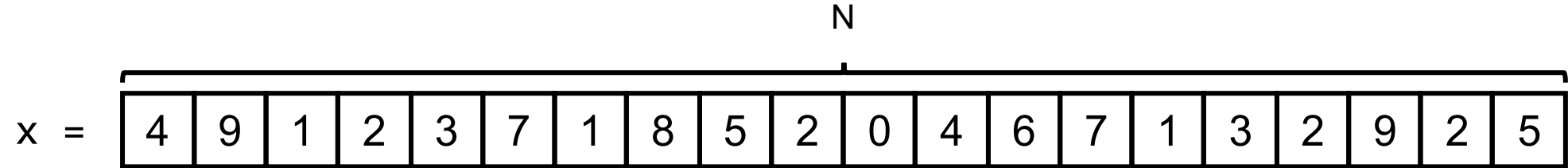
1. Parallelism in general
2. Parallelism in Python
3. Parallelism in practice (i.e., how to make it scale)

Parallelism

What is parallelism?

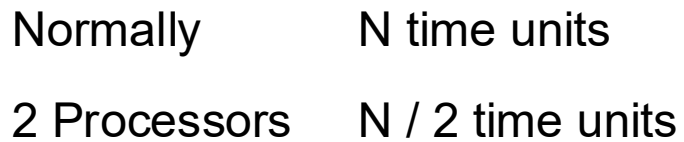
When several operations are being
done *simultaneously*

Parallelism

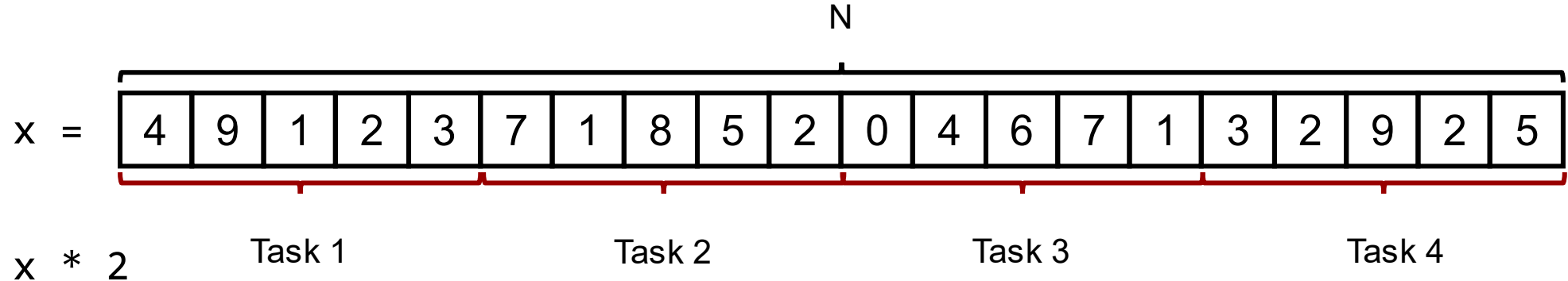


x * 2

Normally N time units



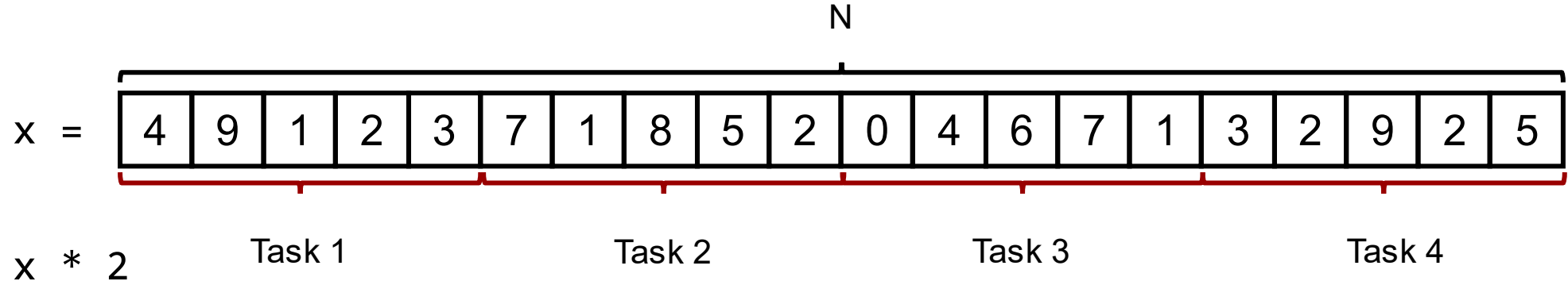
Parallelism



Normally	N time units
2 Processors	N / 2 time units
4 Processors	N / 4 time units

Embarrassingly parallel:
Every unit of work is independent.
I.e., everything can be parallelized.

Parallelism



Normally	N time units	1x
2 Processors	N / 2 time units	2x
4 Processors	N / 4 time units	4x

Speed-up:

$$S(n) = \frac{T(1)}{T(n)}$$

Wall-clock time



Parallelism in Python

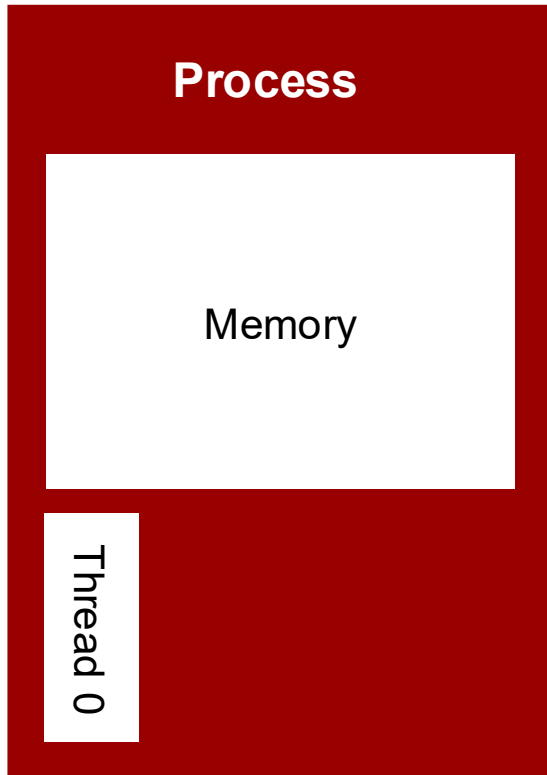
Multi-threading & Multi-processing

Parallelism in Python

~~Multi-threading~~
&
Multi-processing

*GIL says NO!!!
...or?*

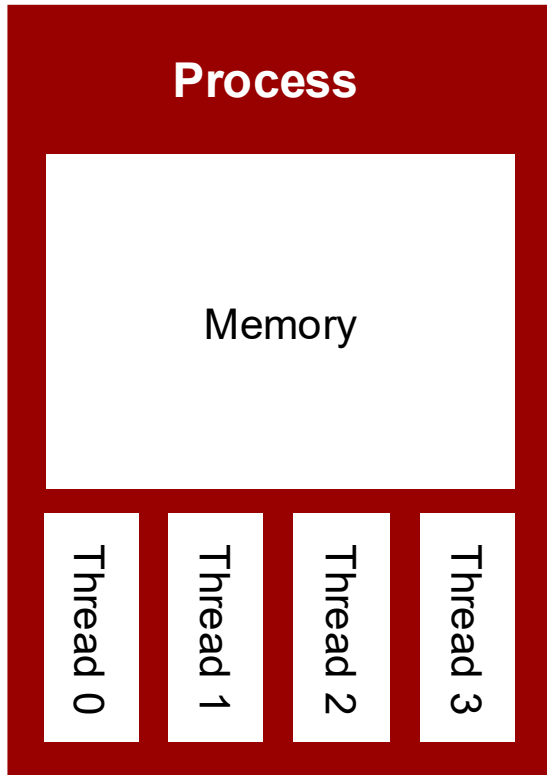
Threads and processes



Process = everything needed to execute a program.

- Memory address space
- Handles to objects
- Etc.
- At least one **thread** of execution

Threads and processes



Process = everything needed to execute a program.

- Memory address space
- Handles to objects
- Etc.
- At least one **thread** of execution

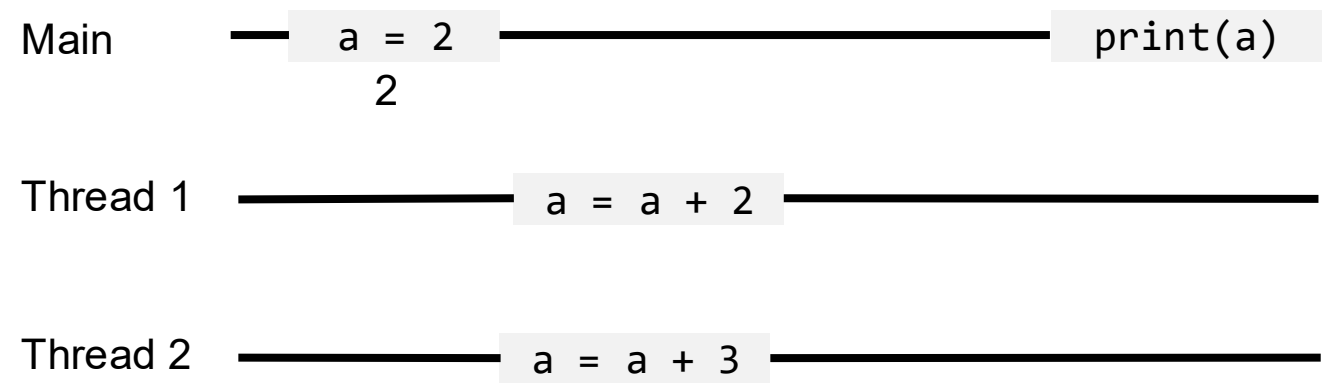
Thread = entity *within* a process to be scheduled for execution.

- Thread local storage and state
- Access to process memory ← **Shared Memory**
- Multiple processors = multiple threads can run in parallel

Threads and processes

```
<< Main >>  
a = 2  
  
<< Thread 1 >>  
a = a + 2  
  
<< Thread 2 >>  
a = a + 3  
  
<< Main >>  
wait_for_threads()  
print(a)
```

What does this print?



Example from: <https://python.land/python-concurrency/the-python-gil>

Threads and processes

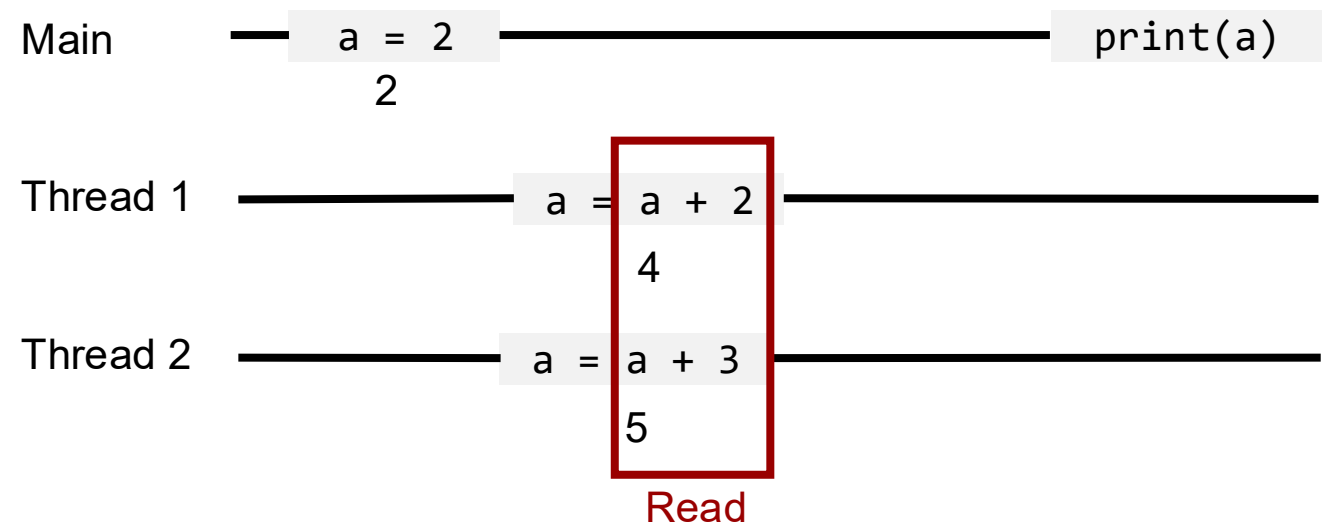
```
<< Main >>
a = 2

<< Thread 1 >>
a = a + 2

<< Thread 2 >>
a = a + 3

<< Main >>
wait_for_threads()
print(a)
```

What does this print?



Example from: <https://python.land/python-concurrency/the-python-gil>

Threads and processes

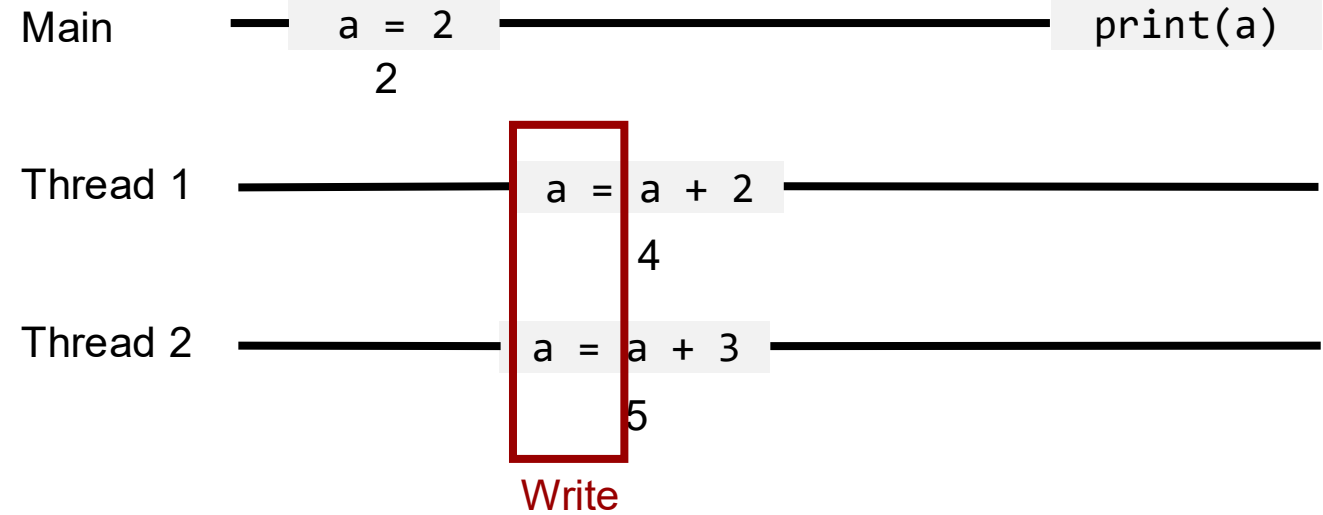
```
<< Main >>
a = 2

<< Thread 1 >>
a = a + 2

<< Thread 2 >>
a = a + 3

<< Main >>
wait_for_threads()
print(a)
```

What does this print?

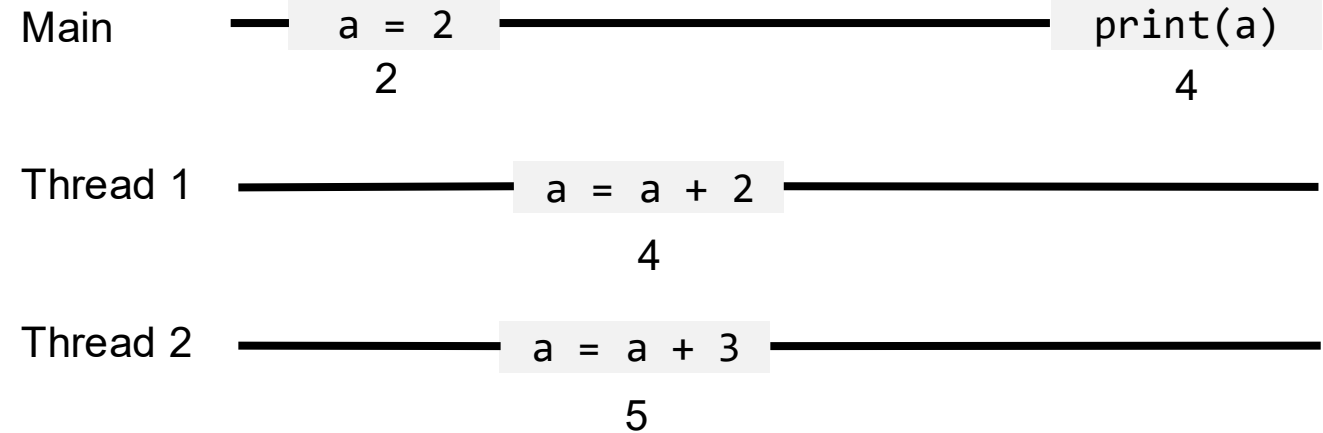


Example from: <https://python.land/python-concurrency/the-python-gil>

Threads and processes

```
<< Main >>  
a = 2  
  
<< Thread 1 >>  
a = a + 2  
  
<< Thread 2 >>  
a = a + 3  
  
<< Main >>  
wait_for_threads()  
print(a)
```

What does this print?



Race condition!

Example from: <https://python.land/python-concurrency/the-python-gil>

Threads and processes

```
<< Main >>  
a = 2
```

```
<< Thread 1 >>  
a = a + 2
```

```
<< Thread 2 >>  
a = a + 3
```

```
<< Main >>  
wait_for_threads()  
print(a)
```

Main

a = 2
2

print(a)
4

Thread 1

a = a + 2
4

a = a + 3

GIL to the rescue!

Race condition

What does this print?

Example from: <https://python.land/python-concurrency/the-python-gil>

GIL: Global Interpreter Lock

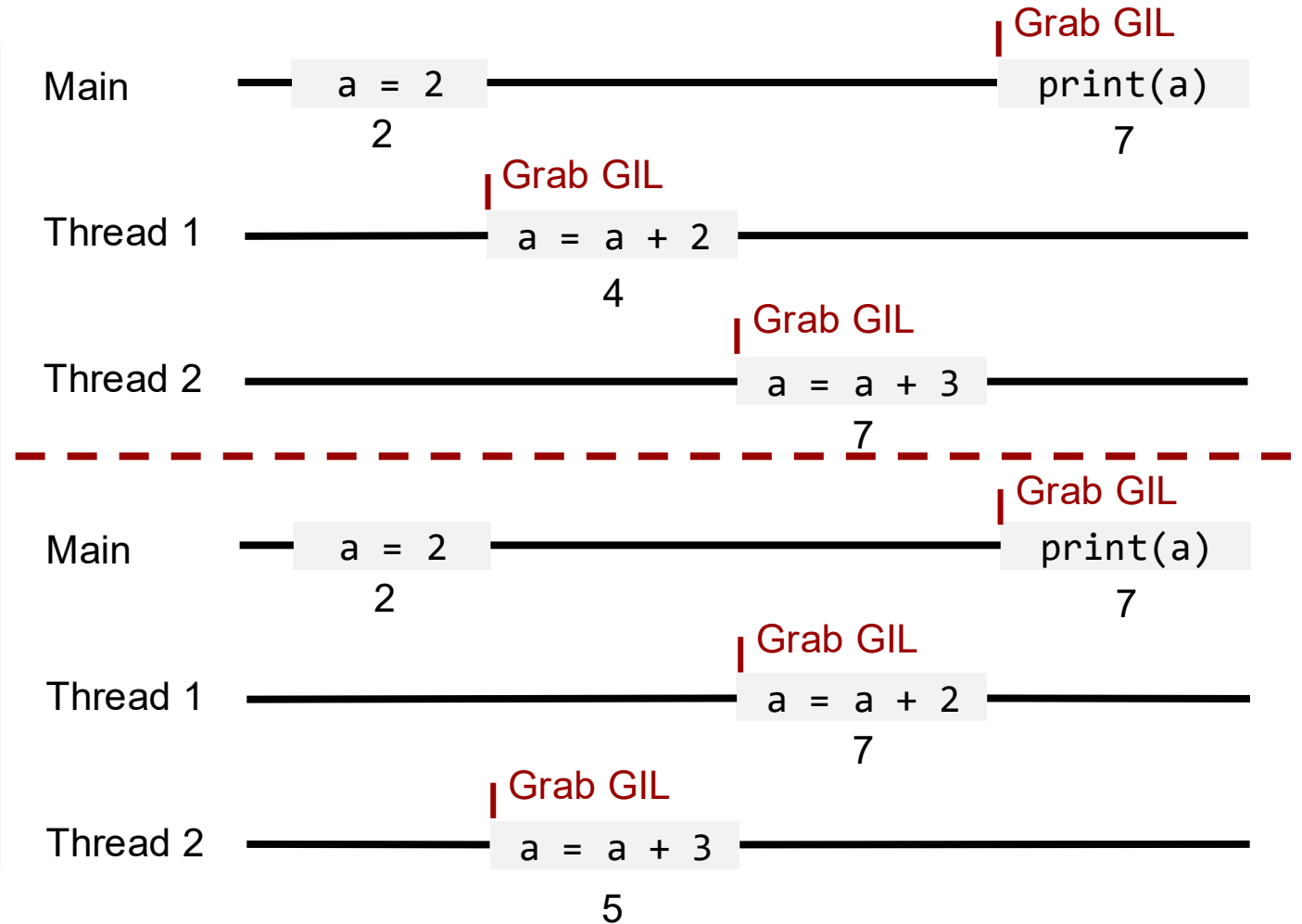
```
<< Main >>
a = 2
```

```
<< Thread 1 >>
a = a + 2
```

```
<< Thread 2 >>
a = a + 3
```

```
<< Main >>
wait_for_threads()
print(a)
```

What does this print?



Example from: <https://python.land/python-concurrency/the-python-gil>

GIL: Global Interpreter Lock

Pro:

- Ensures no race conditions – convenient!

Con:

- Compute heavy multi-threaded = impossible

Some operations manually release the GIL:

- File reading
- I/O
- NumPy

Multi-threading

```
import numpy as np

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

for a in arr_x10:
    np.sum(a)
```

sums.py

```
$ time python sums.py # Single thread
```

Multi-threading

```
import numpy as np

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

for a in arr_x10:
    np.sum(a)
```

sums.py

```
$ time python sums.py # Single thread

real    0m3.634s ← Wall-clock time
user    0m3.573s ← CPU time
sys     0m0.036s ←
```

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(2) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread

real    0m3.634s ← Wall-clock time
user    0m3.573s ← CPU time
sys     0m0.036s ←
$ time python sums.py # 2 threads
```

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(2) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread

real    0m3.634s ← Wall-clock time
user    0m3.573s
sys     0m0.061s
$ t: ThreadPool(2)
```

Task queue

Thread 1

Thread 2

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(2) as pool:
    pool.map(np.sum, arr_x10)
```



sums.py

```
$ time python sums.py # Single thread
```

```
real    0m3.634s ← Wall-clock time
```

```
user    0m3.573s
```

```
sys     0m0.061s
```

```
$ t:
```

Task queue

np.sum(arr_x10[0])

np.sum(arr_x10[1])

np.sum(arr_x10[2])

np.sum(arr_x10[3])

Thread 1

Thread 2

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(2) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread

real    0m3.634s ← Wall-clock time
user    0m3.573s
sys     0m0.061s
$ t:
```

ThreadPool(2)

Task queue

np.sum(arr_x10[0])

np.sum(arr_x10[1])

np.sum(arr_x10[2])

np.sum(arr_x10[3])

Thread 1

Thread 2

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(2) as pool:
    for a in arr_x10:
        # Add to task queue
        pool.apply_async(np.sum, (a,))
    pool.close() # No more tasks
    pool.join()  # Wait for completion
```

sums.py

```
$ time python sums.py # Single thread
```

```
real    0m3.634s ← Wall-clock time
```

```
user    0m3.573s
```

```
sys     0m0.000s
```

```
$ t:
```

ThreadPool(2)

Task queue

np.sum(arr_x10[0])

np.sum(arr_x10[1])

np.sum(arr_x10[2])

np.sum(arr_x10[3])

Thread 1

Thread 2

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(2) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread

real    0m3.634s ← Wall-clock time
user    0m3.573s ← CPU time
sys     0m0.036s ←
$ time python sums.py # 2 threads

real    0m1.873s
user    0m3.179s
sys     0m0.047s

Speed-up:
3.624 / 1.873 = 1.94
```


Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(4) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread

real    0m3.634s ← Wall-clock time
user    0m3.573s ← CPU time
sys     0m0.036s ←
$ time python sums.py # 2 threads

real    0m1.873s      Speed-up:
user    0m3.179s      3.624 / 1.873 = 1.94
sys     0m0.047s
$ time python sums.py # 4 threads

real    0m1.136s      Speed-up:
user    0m3.282s      3.624 / 1.136 = 3.20
sys     0m0.055s
```

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(4) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py

Main

— arr = ... —

Thread 1

—

Thread 2

—

Thread 3

—

Thread 4

—

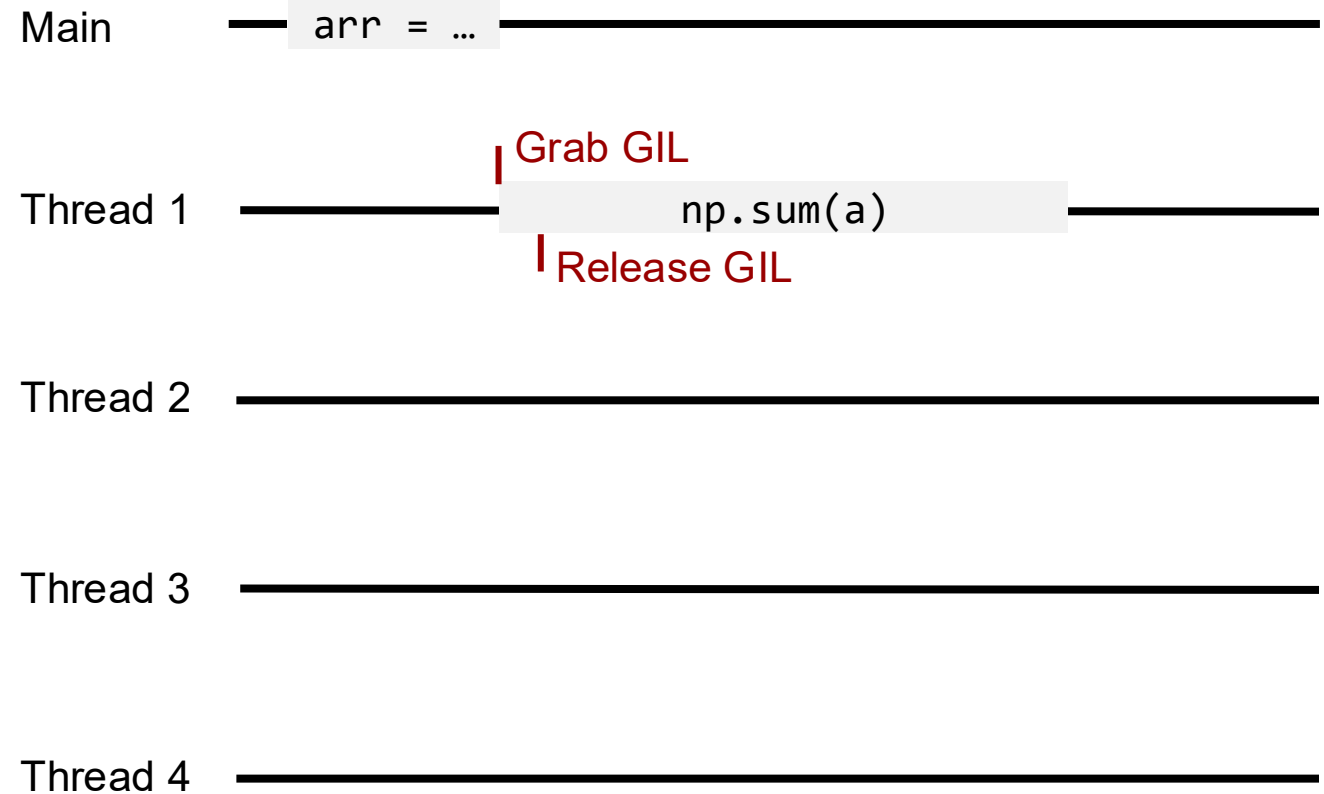
Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(4) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py



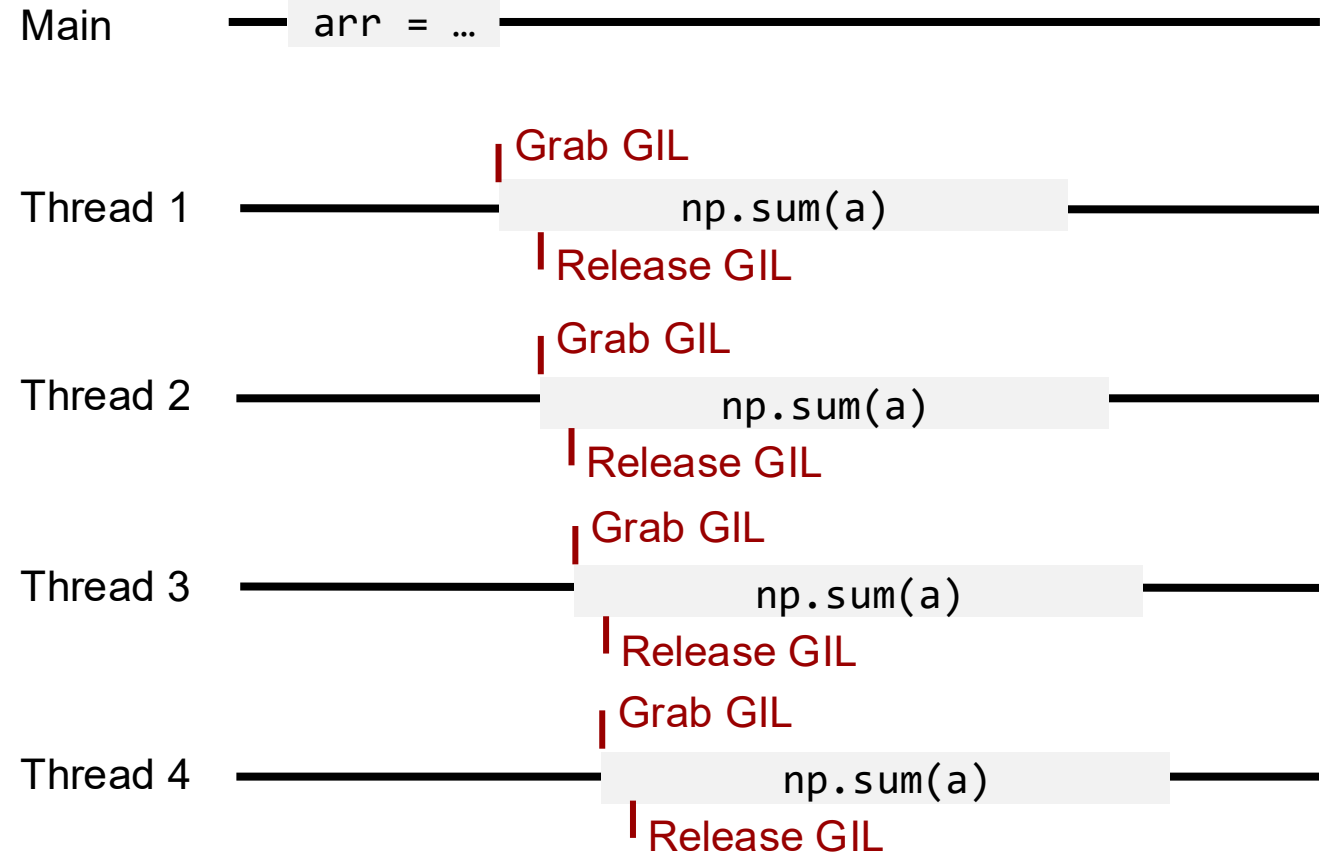
Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(4) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py



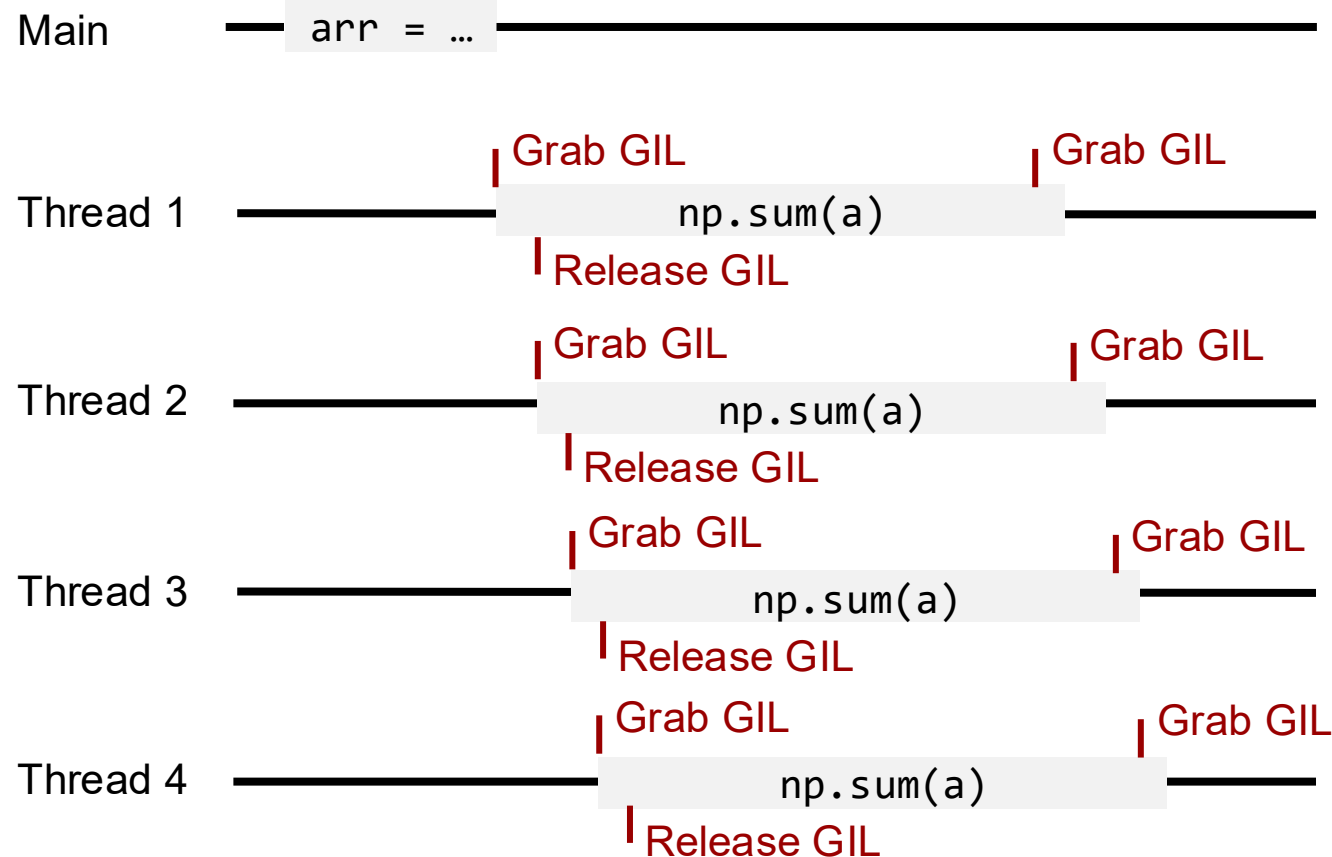
Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

with ThreadPool(4) as pool:
    pool.map(np.sum, arr_x10)
```

sums.py



Multi-threading

```
import numpy as np

arr = [0] * 10_000_000
arr_x10 = [arr] * 10

def manual_sum(arr):
    s = 0
    for a in arr:
        s += a
    return s

for a in arr_x10:
    manual_sum(a)
```

sums.py

```
$ time python sums.py # Single thread
```

real	0m1.916s	← Wall-clock time
user	0m1.800s	← CPU time
sys	0m0.069s	← CPU time

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = [0] * 10_000_000
arr_x10 = [arr] * 10

def manual_sum(arr):
    s = 0
    for a in arr:
        s += a
    return s

with ThreadPool(2) as pool:
    pool.map(manual_sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread

real    0m1.916s ← Wall-clock time
user    0m1.800s ← CPU time
sys     0m0.069s ←
$ time python sums.py # 2 threads

real    0m1.855s
user    0m1.790s
sys     0m0.048s
```

Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = [0] * 10_000_000
arr_x10 = [arr] * 10

def manual_sum(arr):
    s = 0
    for a in arr:
        s += a
    return s

with ThreadPool(4) as pool:
    pool.map(manual_sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread

real    0m1.916s ← Wall-clock time
user    0m1.800s ← CPU time
sys     0m0.069s ←
$ time python sums.py # 2 threads

real    0m1.855s
user    0m1.790s
sys     0m0.048s
$ time python sums.py # 4 threads

real    0m3.254s
user    0m3.157s
sys     0m0.075s
```


Multi-threading

```
import numpy as np
from multiprocessing.pool import ThreadPool
```

```
arr = [0] * 10_000_000
arr_x10 =
```

```
def manual_sum(s):
    for i in s:
        i *= 10
    return s
```

```
with ThreadPool(4) as pool:
    pool.map(manual_sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread
```

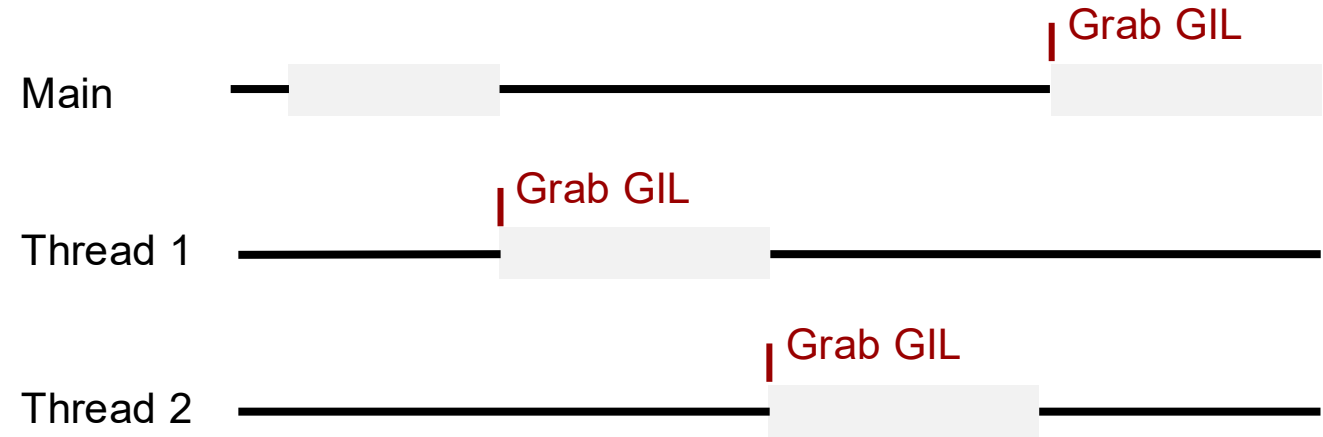
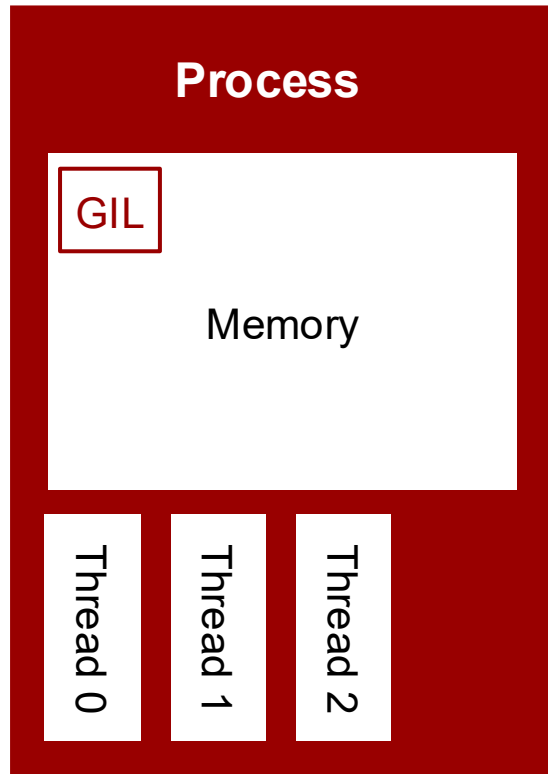
real	0m1.916s	← Wall-clock time
user	0m1.800s	← CPU time
sys	0m0.069s	← CPU time

```
... # 2 threads
```

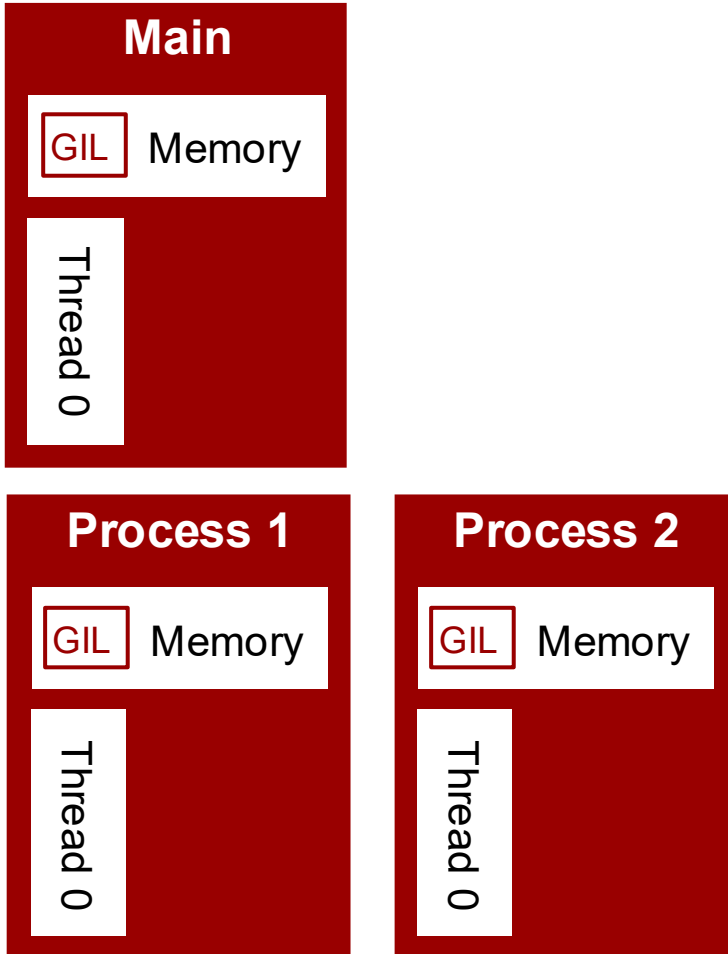
Moral: multi-threading *only* works if threads almost never hold the GIL

real	0m3.254s
user	0m3.157s
sys	0m0.075s

Multi-processing



Multi-processing



Pro:

Processes can always run in parallel

Cons:

Processes have more overhead

No shared memory – must explicitly copy

Multi-processing

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = [0] * 10_000_000
arr_x10 = [arr] * 10

def manual_sum(arr):
    s = 0
    for a in arr:
        s += a
    return s

with ThreadPool(2) as pool:
    pool.map(manual_sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread

real    0m1.793s
user    0m1.716s
sys     0m0.048s
```

Multi-processing

```
import numpy as np
from multiprocessing.pool import Pool

arr = [0] * 10_000_000
arr_x10 = [arr] * 10

def manual_sum(arr):
    s = 0
    for a in arr:
        s += a
    return s

with Pool(2) as pool:
    pool.map(manual_sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread
```

```
real    0m1.793s
user    0m1.716s
sys     0m0.048s
```

Multi-processing

```
import numpy as np
from multiprocessing.pool import Pool

arr = [0] * 10_000_000
arr_x10 = [arr] * 10

def manual_sum(arr):
    s = 0
    for a in arr:
        s += a
    return s

with Pool(1) as pool:
    pool.map(manual_sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread
```

```
real    0m1.793s
```

```
user    0m1.716s
```

```
sys     0m0.048s
```

```
$ time python sums.py # 1 worker process
```

```
real    0m2.410s
```

```
user    0m2.315s
```

```
sys     0m0.172s
```

Overhead!

Multi-processing

```
import numpy as np
from multiprocessing.pool import Pool

arr = [0] * 10_000_000
arr_x10 = [arr] * 10

def manual_sum(arr):
    s = 0
    for a in arr:
        s += a
    return s

with Pool(2) as pool:
    pool.map(manual_sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread
```

```
real    0m1.793s
user    0m1.716s
sys     0m0.048s
```

```
$ time python sums.py # 1 worker process
```

```
real    0m2.410s
user    0m2.315s
sys     0m0.172s
```

```
$ time python sums.py # 2 worker processes
```

```
real    0m1.620s
user    0m2.299s
sys     0m0.187s
```

Speed-up:
 $2.410 / 1.620 = 1.49$

Multi-processing

```
import numpy as np
from multiprocessing.pool import Pool
```

```
arr = [0] * 10 000_000
arr_x10 =
```

```
def manual_sum(s):
    for i in s:
        s[i] += 1
    return s
```

```
with Pool(2) as pool:
    pool.map(manual_sum, arr_x10)
```

sums.py

```
$ time python sums.py # Single thread
```

```
real    0m1.793s
user    0m1.716s
sys     0m0.048s
```

```
... # 1 worker process
```

Moral: multi-processing has a lot of overhead but *does* work

```
$ time python sums.py
```

```
real    0m1.620s
user    0m2.299s
sys     0m0.187s
```

Speed-up:
 $2.410 / 1.620 = 1.49$

Quiz time!

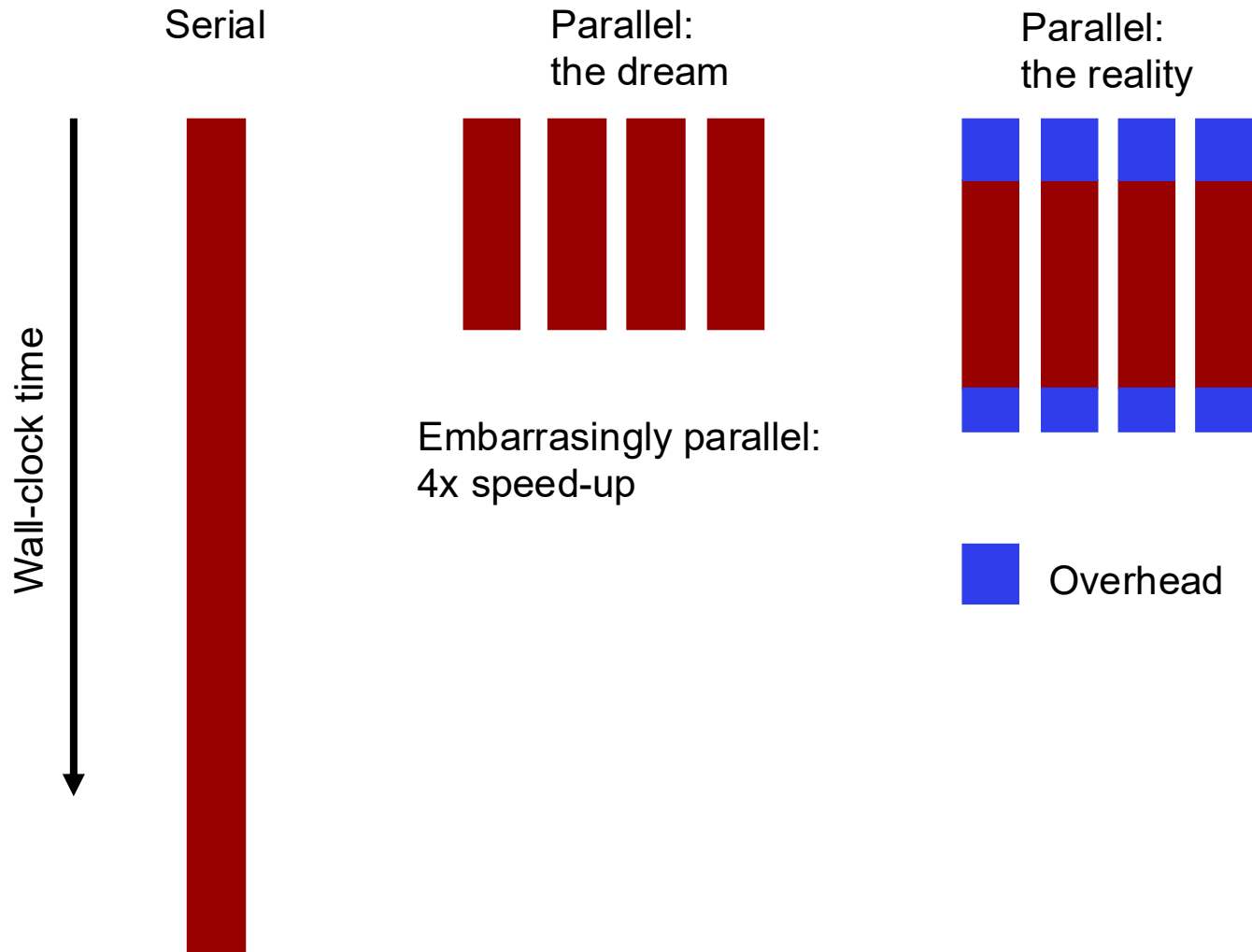
Parallelism in practice

how to make it scale

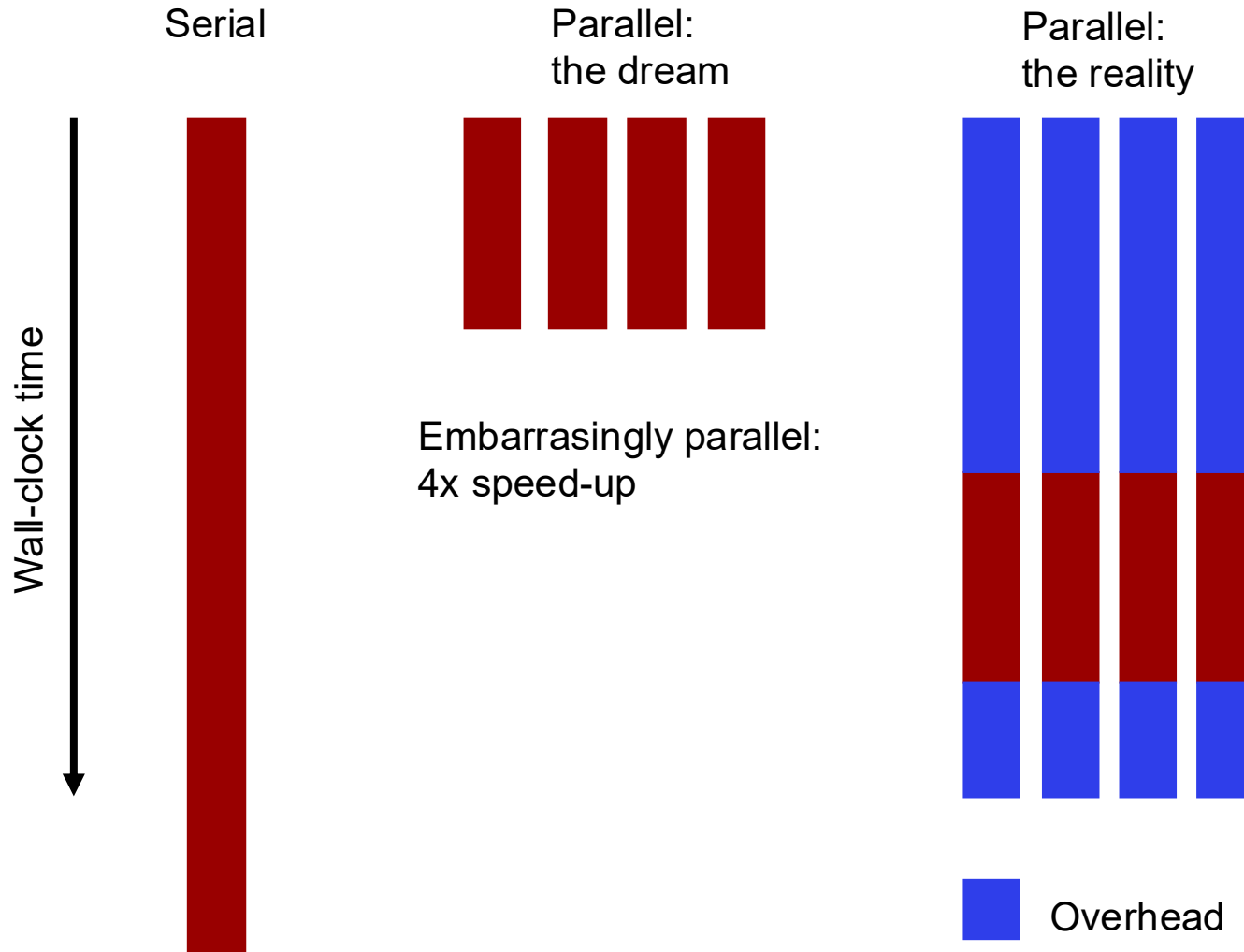
**I just spent 2 days making it parallel,
why the #@! isn't it faster!?**

1. Parallelization overhead
2. Too much communication
3. Load balancing
4. Amdahl's law

Parallelization overhead



Parallelization overhead



Problem:

Parallelization overhead is larger than runtime

Solution:

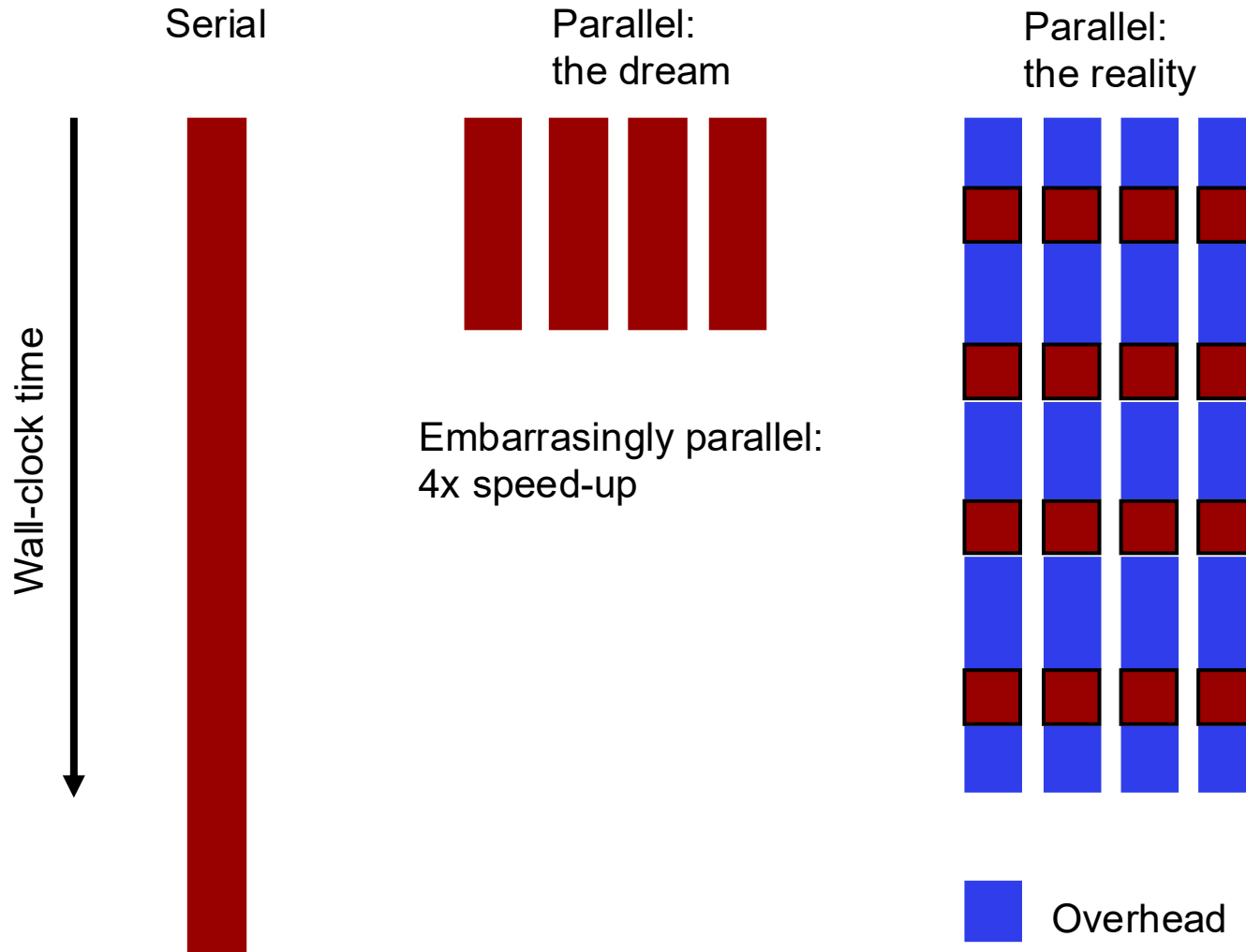
Sorry! ☹️

Escape to C

Find a Python package

overhead = tid worker bruger på at finde ud af hvad den skal lave basically

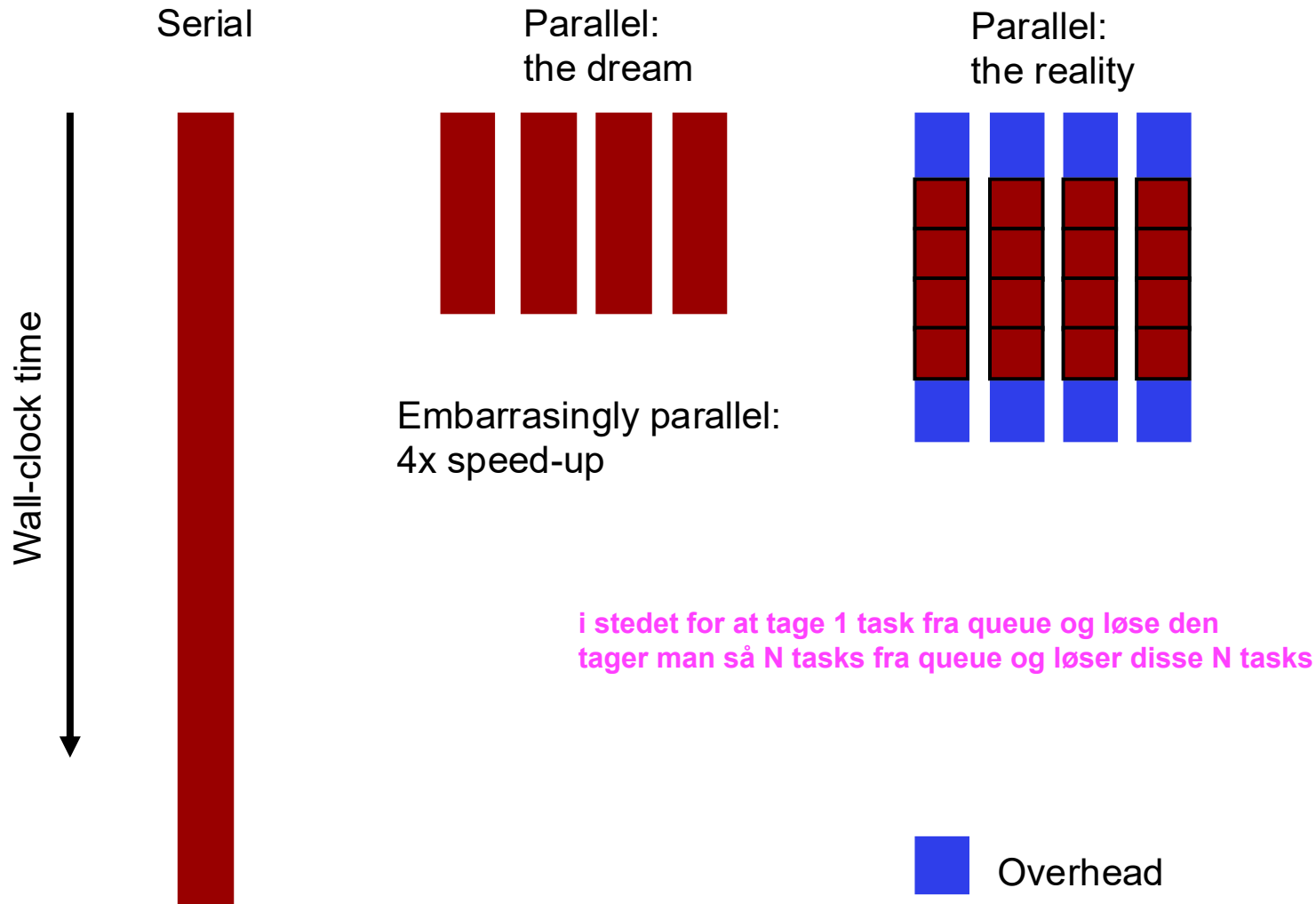
Parallelization overhead



Problem:

Parallelization overhead is larger than runtime

Parallelization overhead



Problem:

Parallelization overhead is larger than runtime

Solution:

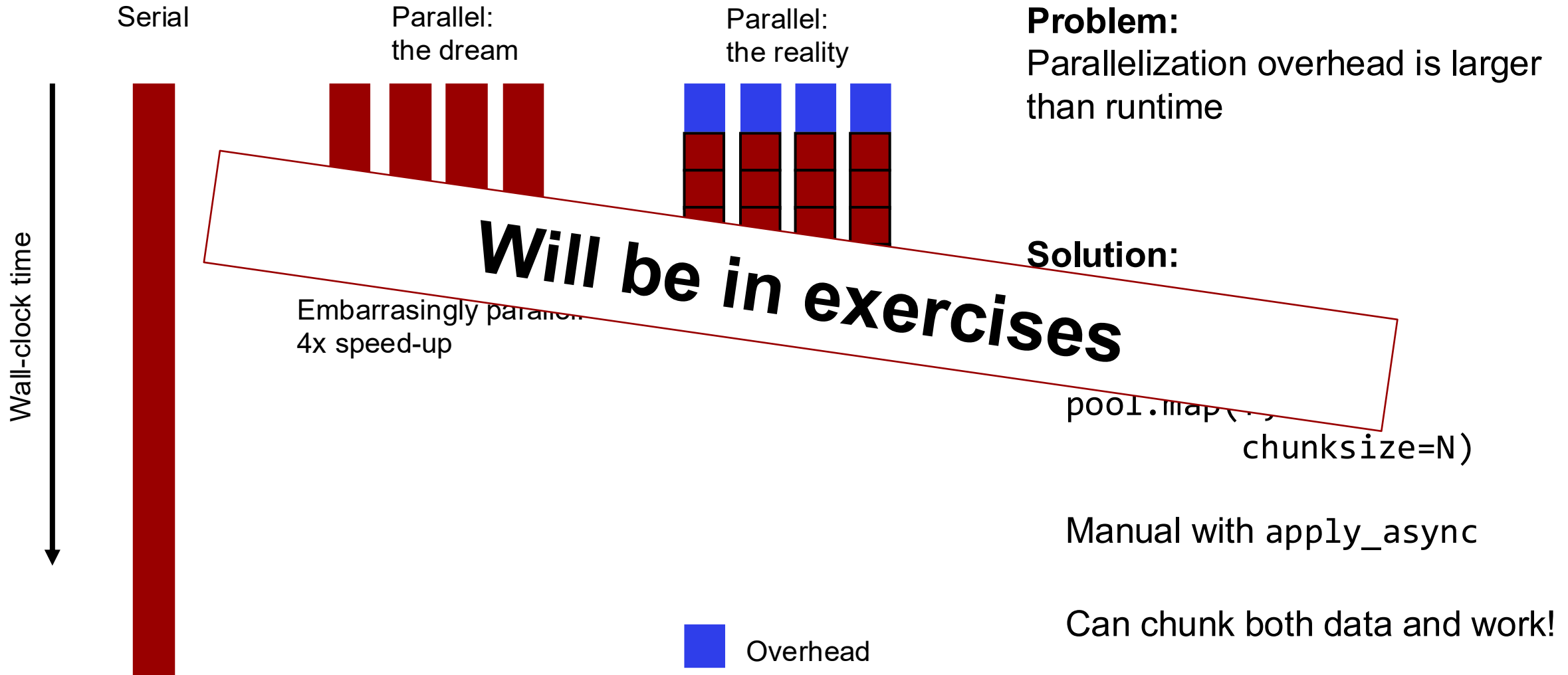
Chunking

```
pool.map(f, arr,  
         chunksize=N)
```

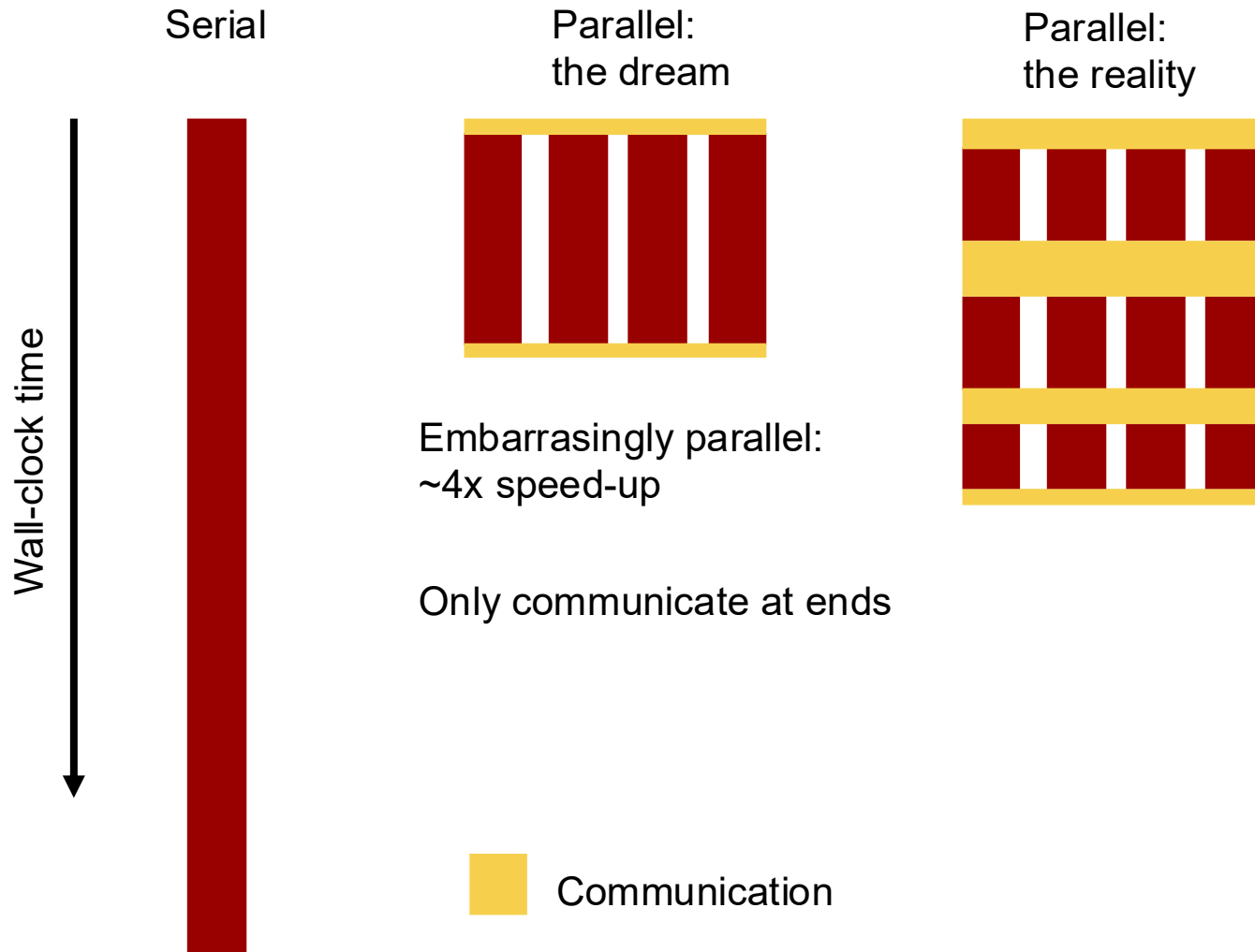
Manual with `apply_async`

Can chunk both data and work!

Parallelization overhead



Too much communication



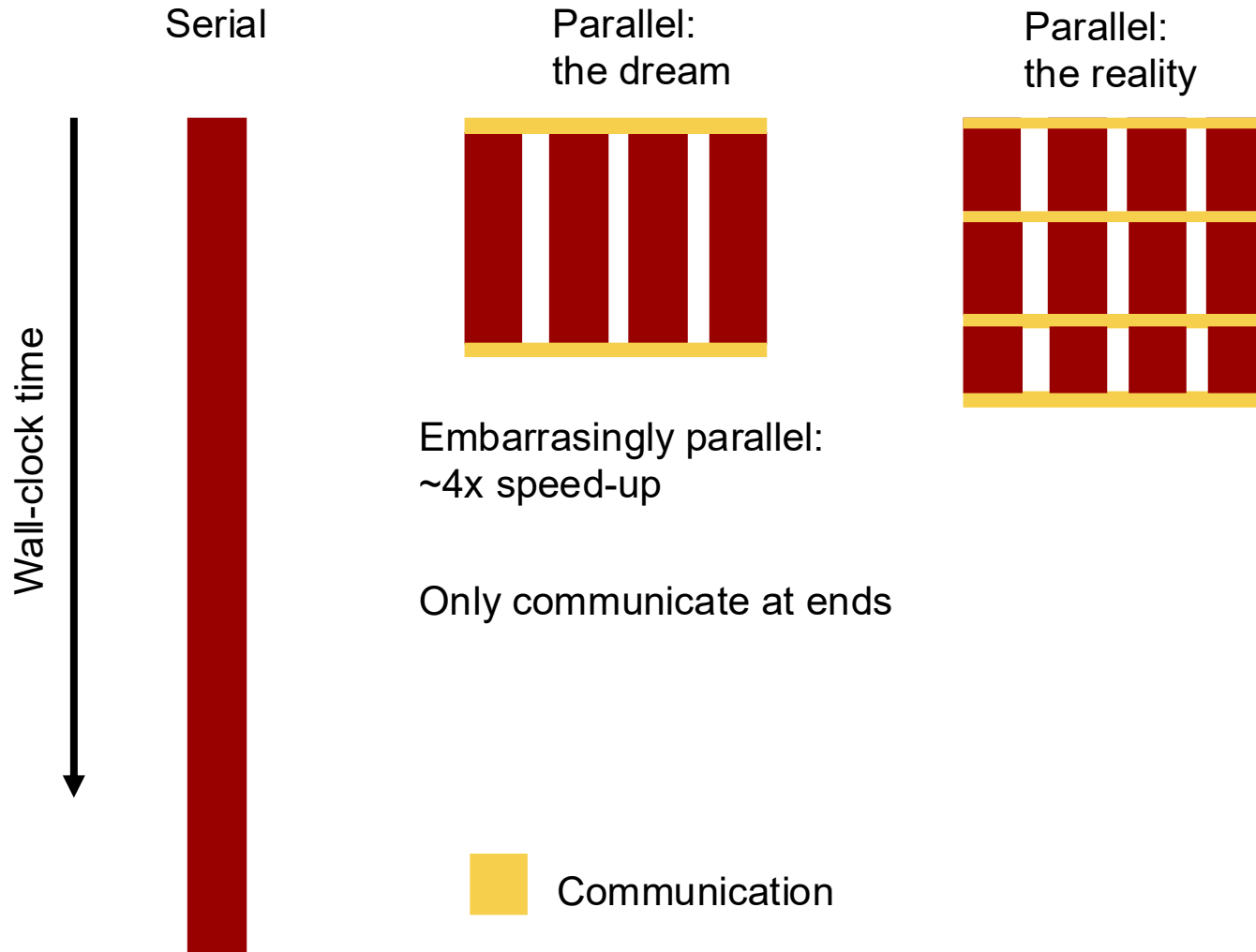
Problem:

Communication adds overhead

Examples:

- Training neural nets on multiple GPUs
- Solving PDEs on large grids with iterative methods
- Particle simulations with attraction (from book)

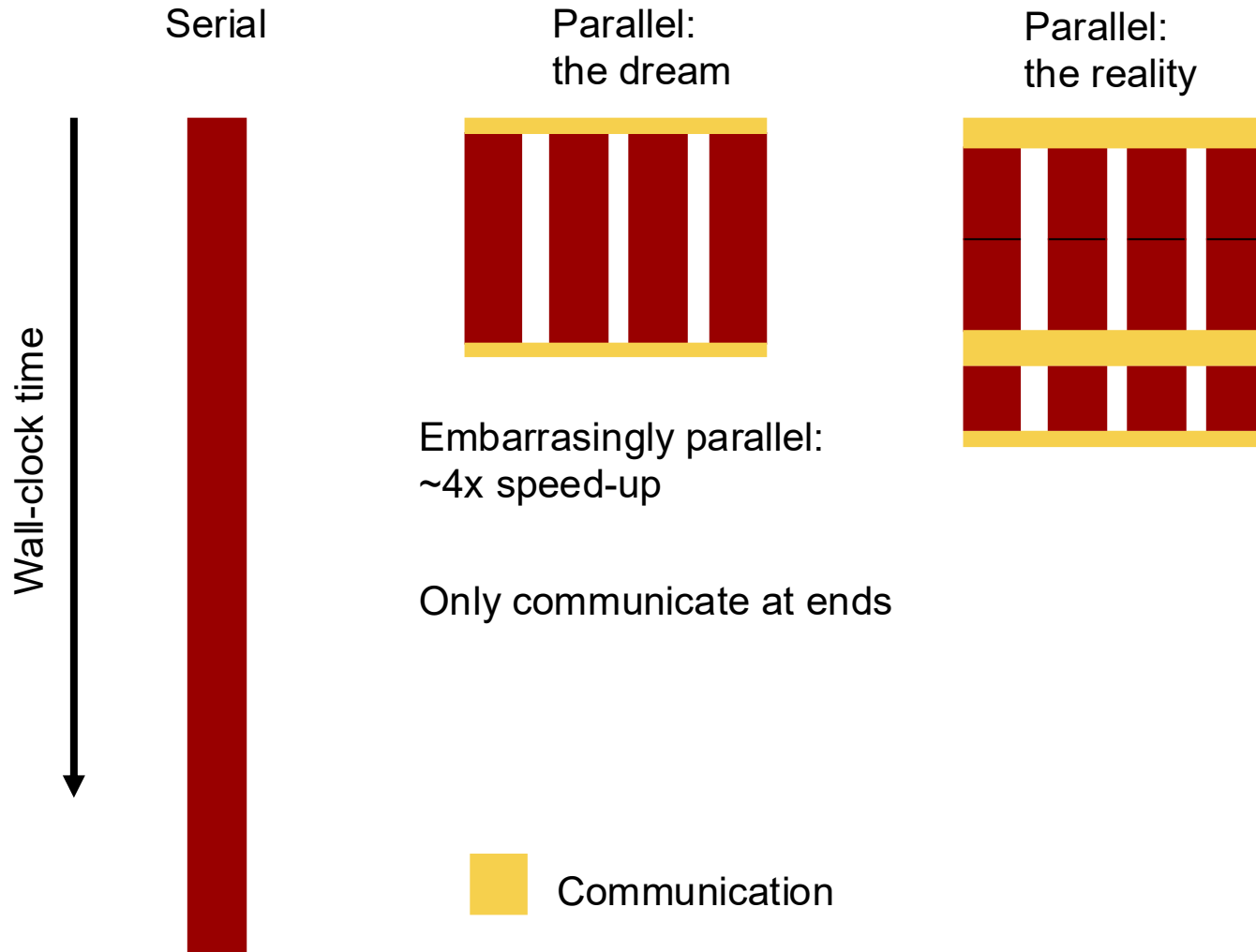
Too much communication



Problem:
Communication adds overhead

Solution:
Send less data

Too much communication



Problem:

Communication adds overhead

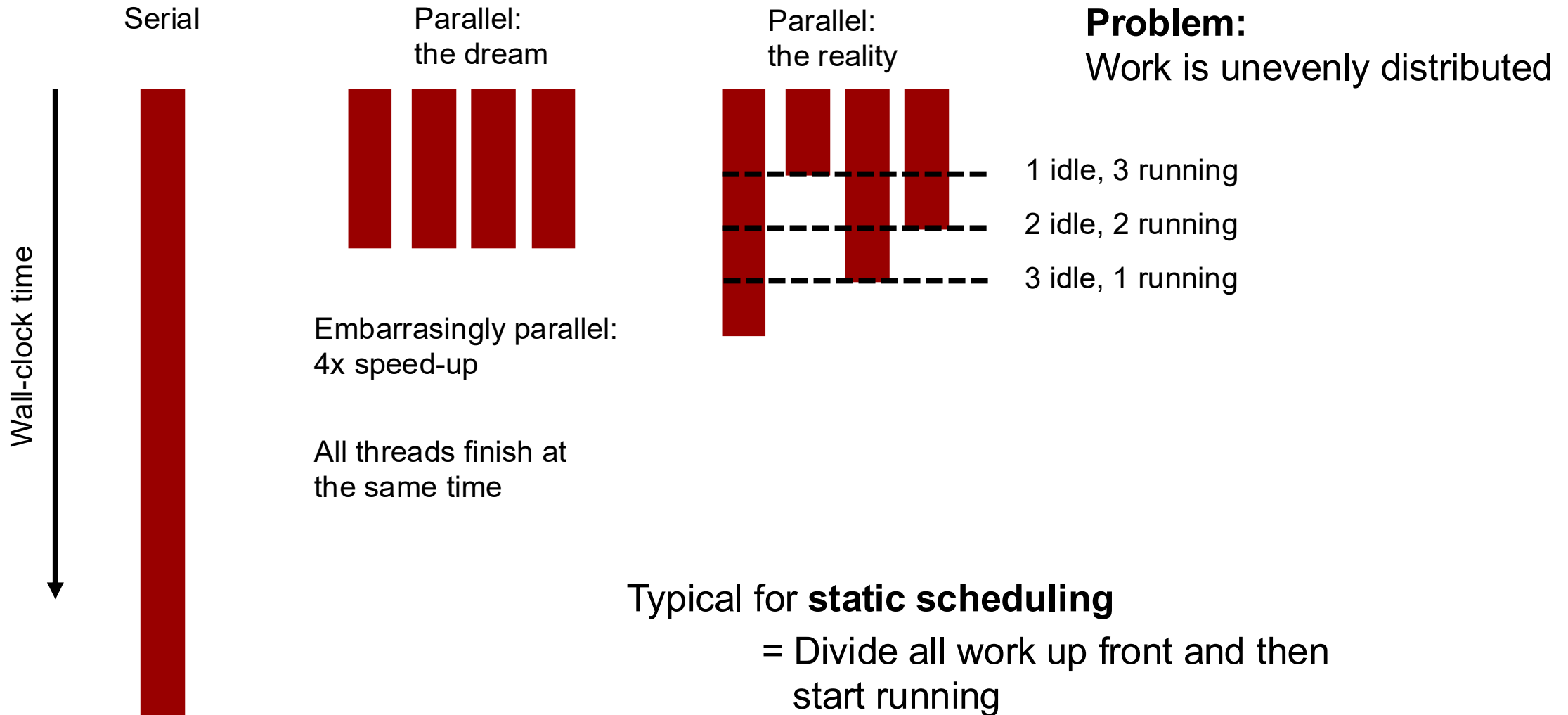
Solution:

Send less data

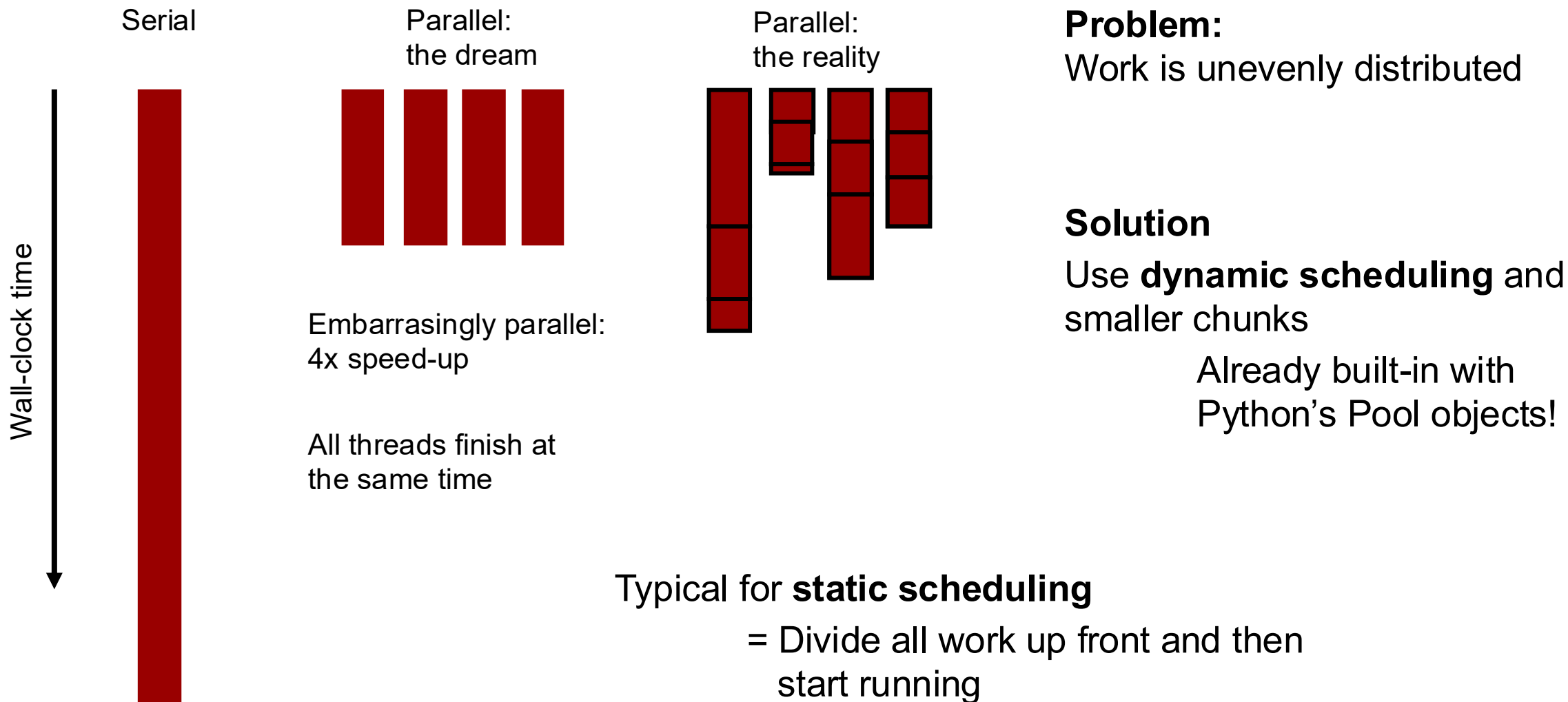
Chunk work / skip communication

WARNING: May *not* be correct!

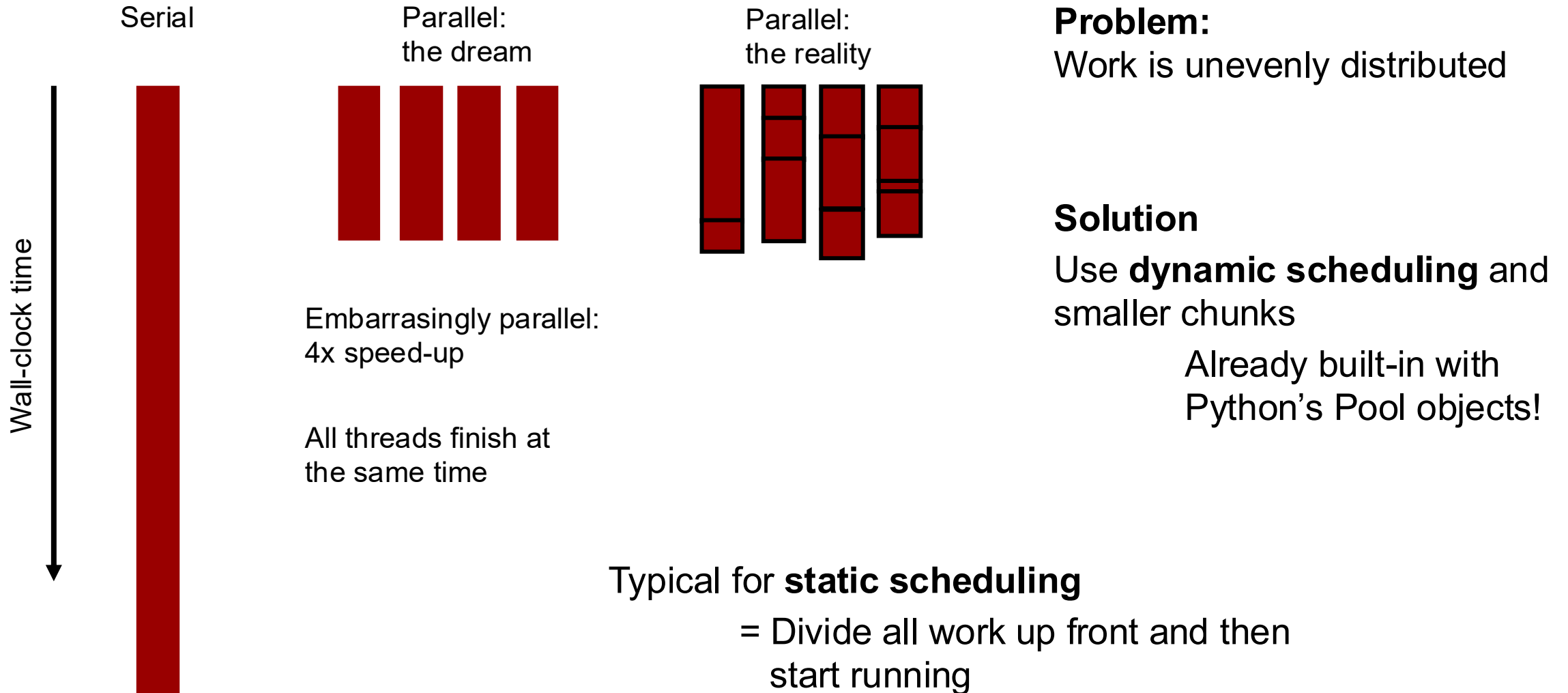
Load balancing



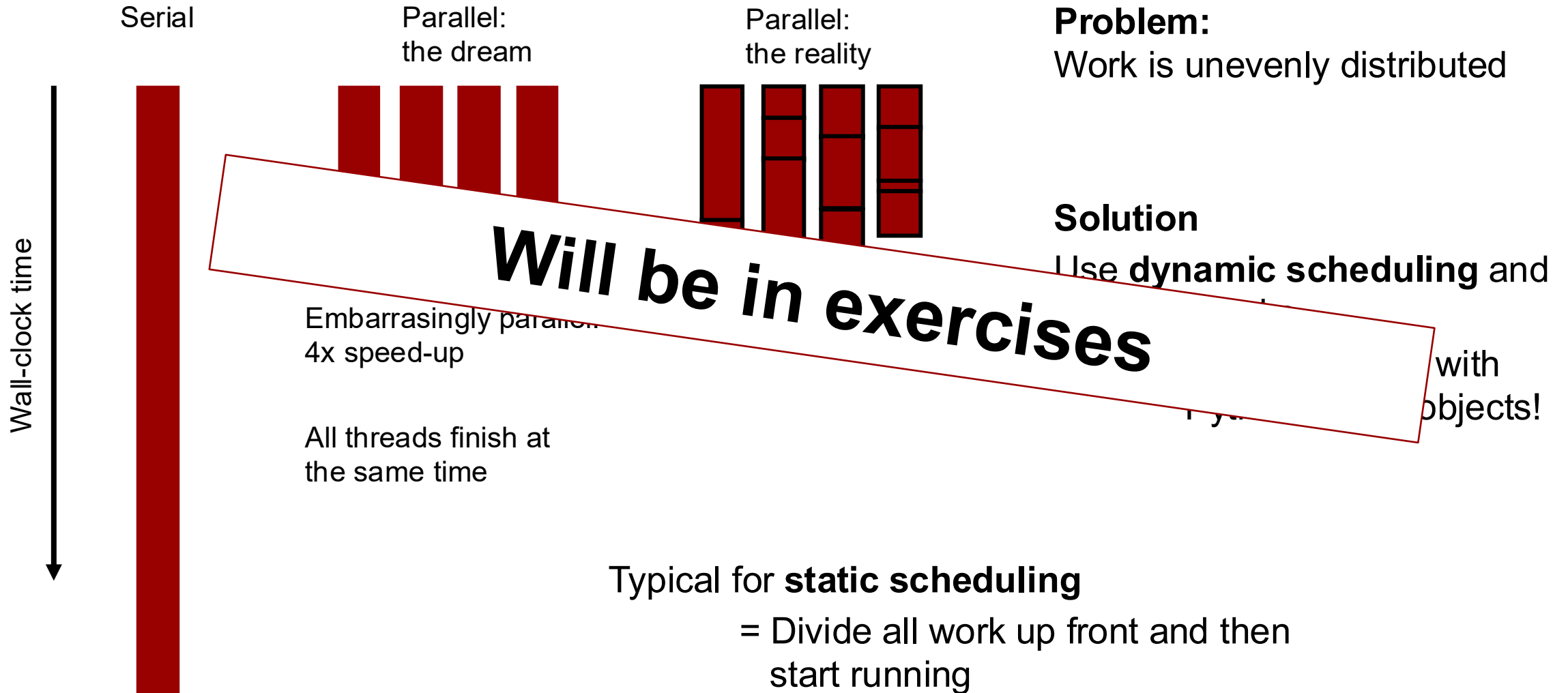
Load balancing



Load balancing



Load balancing



Amdahl's law

```
import numpy as np
from multiprocessing.pool import ThreadPool

arr = np.zeros((1024, 1024, 1024))
arr_x10 = [arr, arr, arr, arr, arr,
           arr, arr, arr, arr, arr]

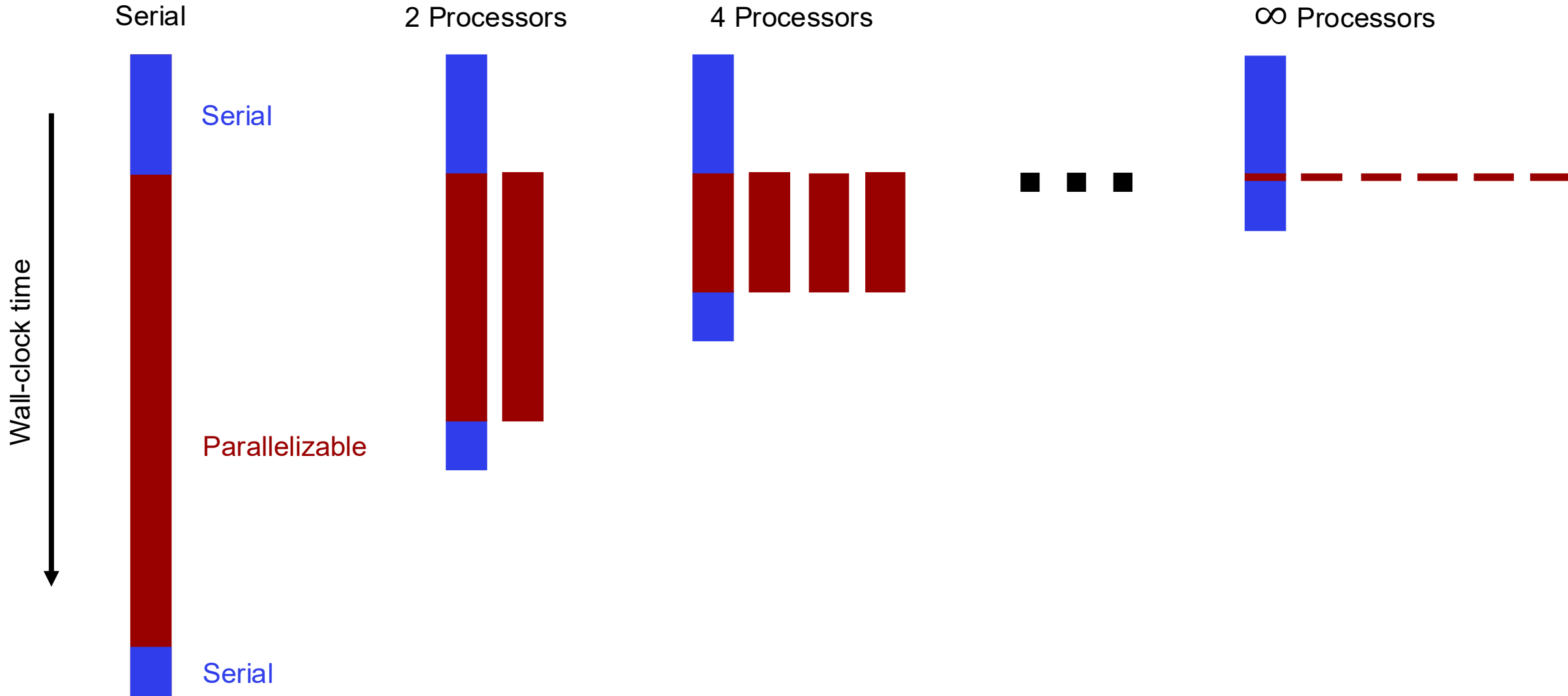
with ThreadPool(4) as pool:
    pool.map(np.sum, arr_x10)
```

} Setup data **Serial**

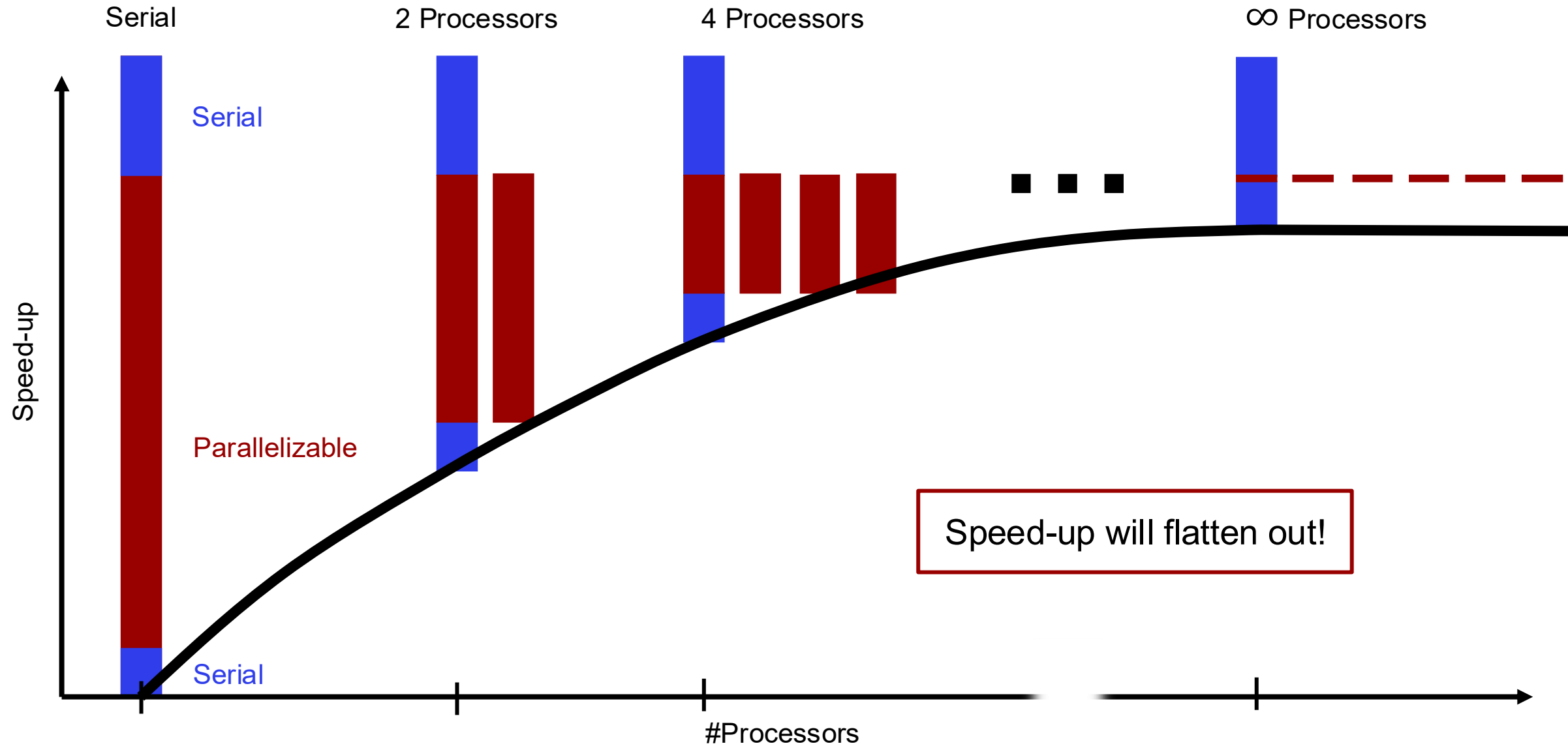
} Compute **Parallel**

sums.py

Amdahl's law



Amdahl's law



Amdahl's law



Serial

Assume our program has a 'parallel fraction' F

Serial execution time:

$$T(1) = (1 - F) * T(1) + F * T(1)$$

Time on p processors:

$$T(p) = (1 - F) * T(1) + \frac{F}{p} * T(1)$$

Parallelizable

Speed-up:

$$S(p) = \frac{T(1)}{T(p)} = \frac{T(1)}{(1-F)*T(1) + \frac{F}{p}*T(1)}$$

Serial

Amdahl's law



Serial

Assume our program has a 'parallel fraction' F

Serial execution time: $T(1) = (1 - F) * T(1) + F * T(1)$

Time on p processors: $T(p) = (1 - F) * T(1) + \frac{F}{p} * T(1)$

Parallelizable

Amdahl's law: $S(p) = \frac{T(1)}{T(p)} = \frac{1}{(1-F) + \frac{F}{p}}$

Serial

Amdahl's law



Serial

Assume our program has a 'parallel fraction' F

Serial execution time: $T(1) = (1 - F) * T(1) + F * T(1)$

Time on p processors: $T(p) = (1 - F) * T(1) + \frac{F}{p} * T(1)$

Parallelizable

Amdahl's law: $S(p) = \frac{T(1)}{T(p)} = \frac{1}{(1-F) + \frac{F}{p}}$

For 'serial fraction' $B = 1 - F$: $= \frac{1}{B + \frac{1-B}{p}}$

Serial

Amdahl's law



Serial

Assume our program has a 'parallel fraction' F

Serial execution time: $T(1) = (1 - F) * T(1) + F * T(1)$

Time on p processors: $T(p) = (1 - F) * T(1) + \frac{F}{p} * T(1)$

Parallelizable

Amdahl's law: $S(p) = \frac{T(1)}{T(p)} = \frac{1}{(1-F) + \frac{F}{p}}$

Limit:

$$S(\infty) = \frac{T(1)}{T(\infty)} = \frac{1}{1-F} = \frac{1}{B}$$

Serial

Amdahl's law

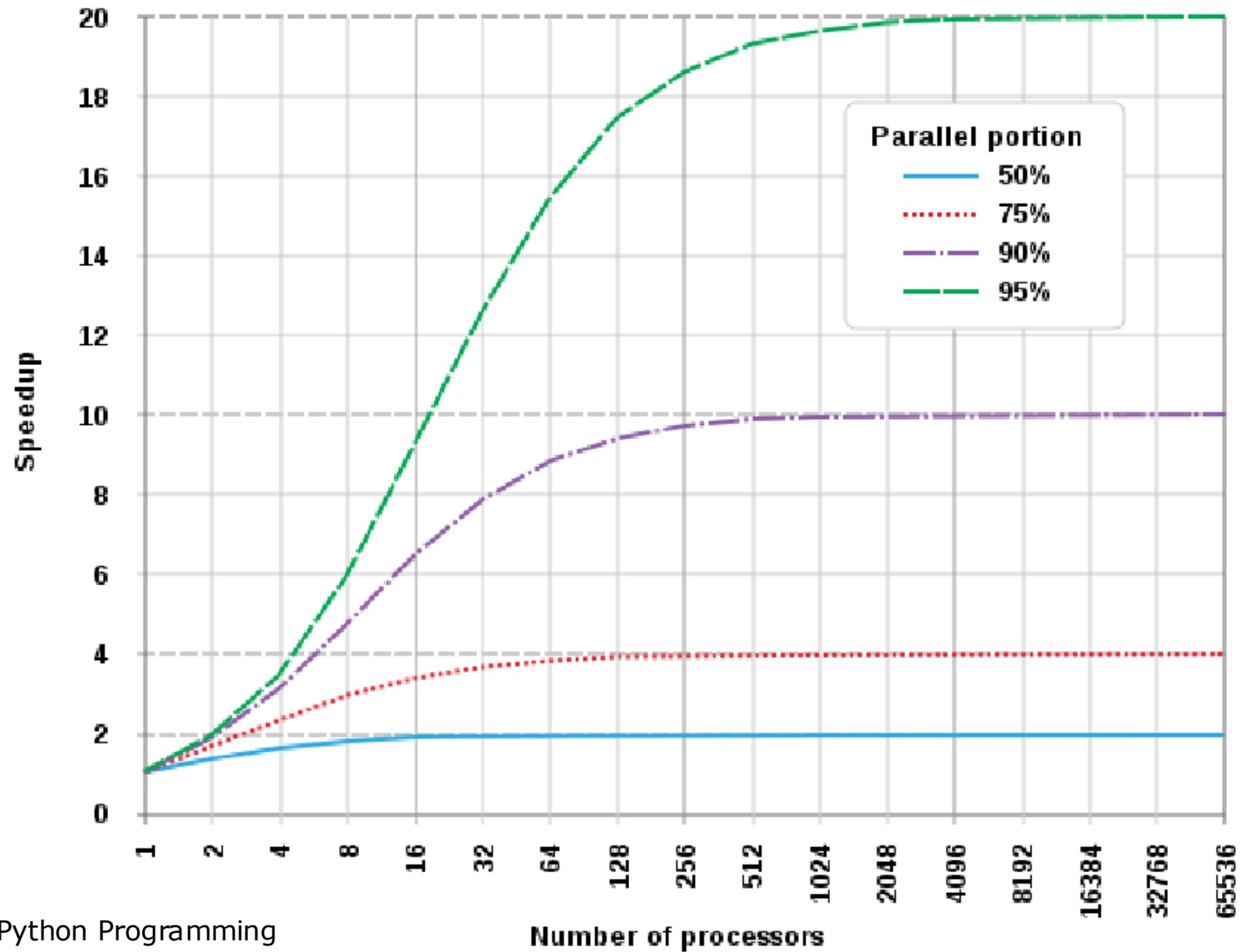


Figure from Advanced Python Programming

Amdahl's law

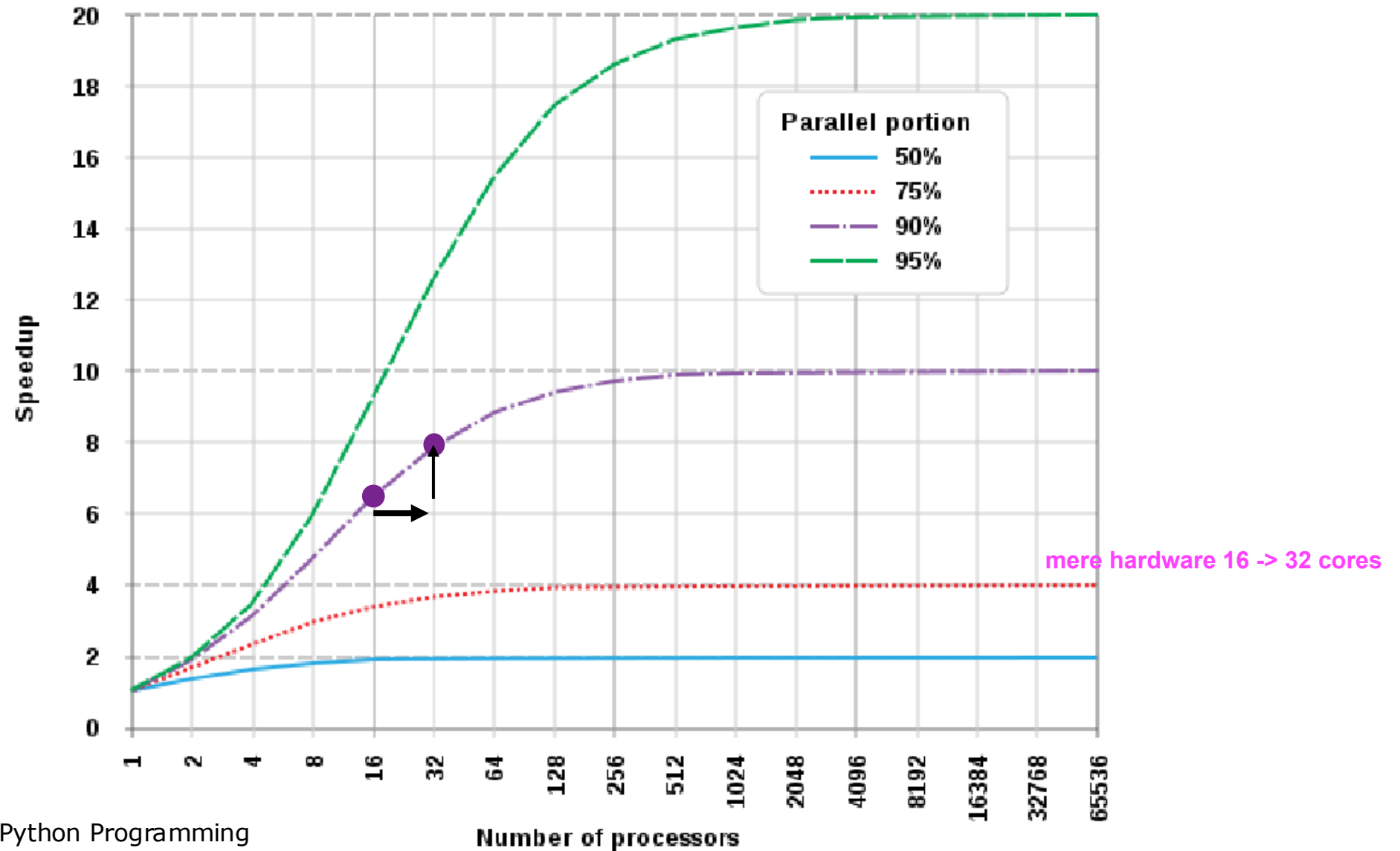
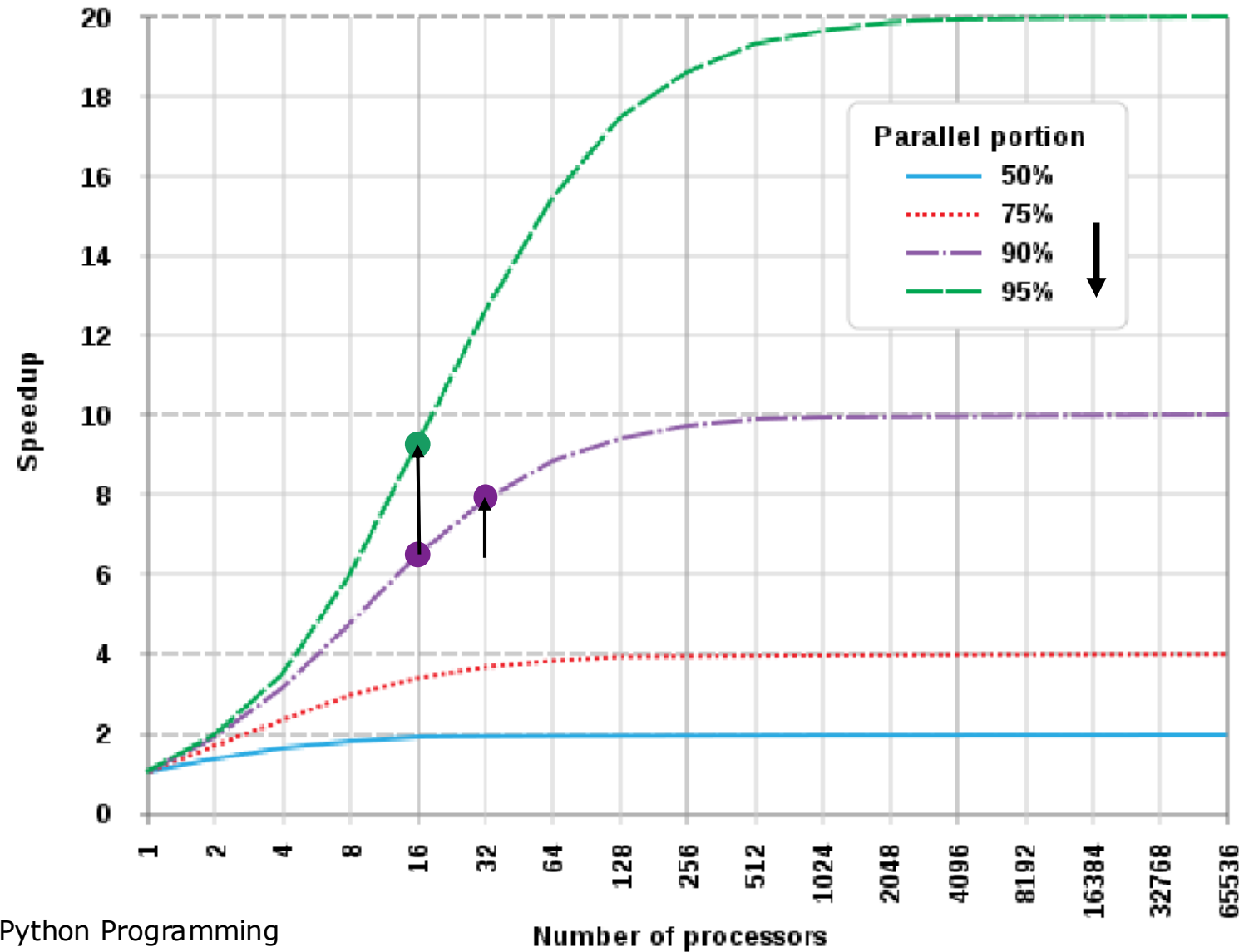


Figure from Advanced Python Programming

Amdahl's law



samme hardware
med større parallel
portion -> øger
speedup

Figure from Advanced Python Programming

Amdahl's law

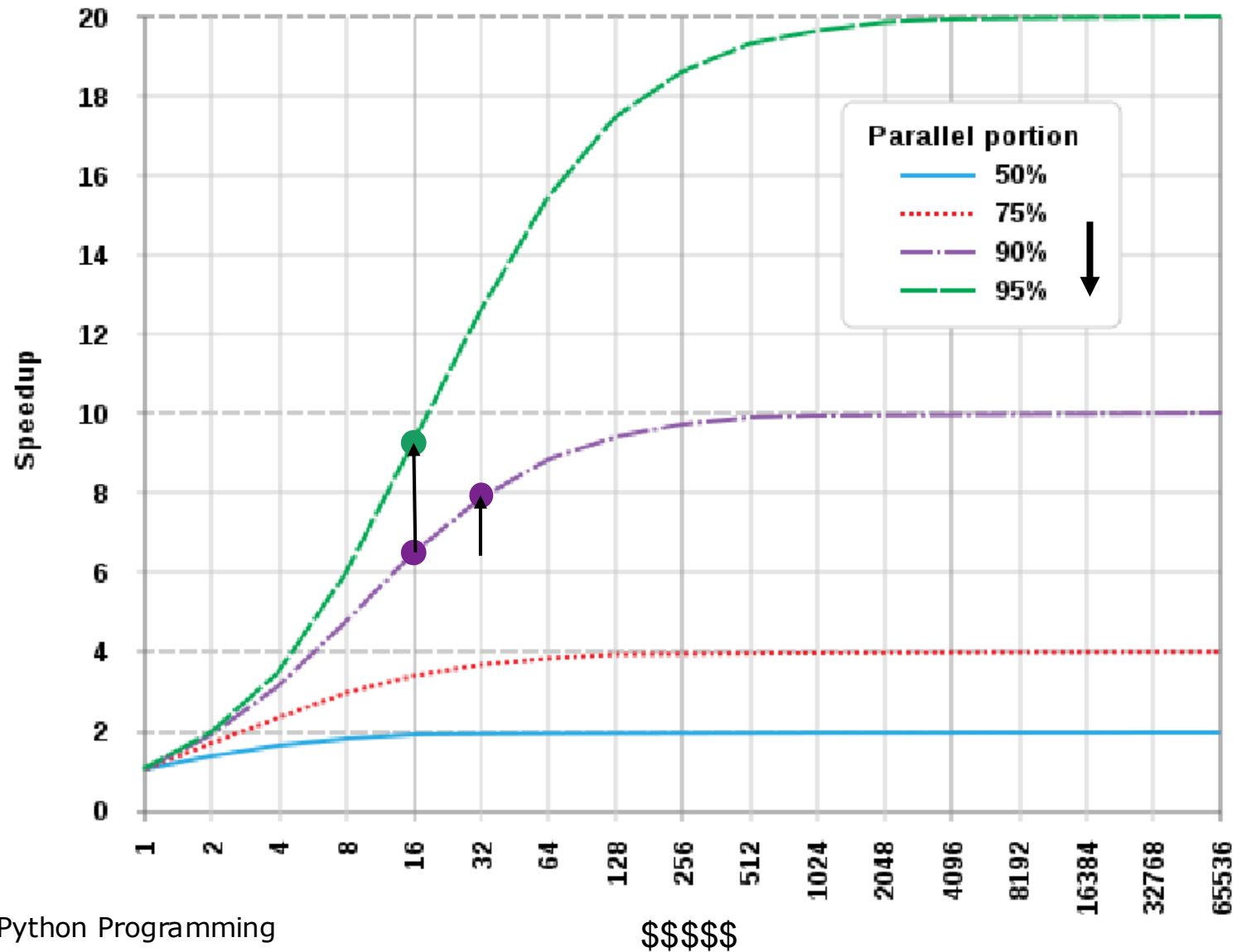


Figure from Advanced Python Programming

Quiz time!

Takeaways

- Parallelization is the name of the game now!
- Multi-threading *can* work, but multi-processing is the general way
- Efficient parallel programs are not trivial – no free lunch!
- Amdahl's law tell us the limits

Today's exercise

Run stuff in parallel and efficiently

- Play with multiprocessing
- Play with chunking
- Play with scheduling
- Play with Amdahl's law
- **NOTE:** These exercises are also for next week.

Useful concepts

Amdahl's law

Parallel fraction F . Serial fraction $B = 1 - F$

$$S(p) = \frac{1}{(1 - F) + \frac{F}{p}} = \frac{1}{B + \frac{1 - B}{p}}$$

$$S(\infty) = \frac{1}{1 - F} = \frac{1}{B}$$

Python Multiprocessing Pool

```
from multiprocessing.pool import Pool  
pool = Pool(n_processes)
```

```
for part in data:  
    pool.apply_async(function, part)
```

```
pool.map(function, data)
```

Change to work node

linuxsh

Submit job script

bsub < submit.sh

Job status

bstat / bjobs

Check job output

bpeek / bpeek <JOBID>

Kill job

bkill <JOBID>

Time command for Python script

time python script.py